

Laboratorio: optimización sin restricciones

Autores (Grupo 4): Javier Blanco Álvarez, Pablo Castañeda Fuentes, Lucía Royo Pascual, Ferrán Soto Moscardó, Jesús Gallinal Moreno.

Asignatura: Optimización

ÍNDICE

RESUMEN.....	1
INTRODUCCIÓN.....	2
Descripción del laboratorio.....	2
Objetivos.....	2
Planteamiento del Problema.....	2
Metodología.....	2
Implementación.....	3
Solución Exacta.....	4
Método de paso fijo.....	6
Partiendo de recta $u(x) = 1$	6
Partiendo de parábola $u(x) = x^2 - x + 1$	20
Método de paso variable.....	34
Gradiente con paso variable para $N = 30$	34
Gradiente con paso variable para distintos puntos.....	35
Análisis y resultados.....	46
Área mínima y el número de puntos N	46
Cálculo del Área mínima y el método de búsqueda de t óptimo.....	48
Estudio del perfil del gradiente.....	49
Para t fijo.....	49
Para t variable.....	51
CONCLUSIONES.....	52
REFERENCIAS.....	53
ANEXOS.....	53
Anexo I: Método Gradiente Paso fijo.....	53
Con aproximación a una recta $u(x) = 1$	53
Con aproximación a una parábola $u(x) = x^2 - x + 1$	53
Anexo II: Función Integral.....	54
Anexo III: Función gradiente.....	54
Anexo III: Método Gradiente Paso variable.....	55
Anexo IV: Métodos de búsqueda para t óptimo.....	60
Fibonacci.....	60
Búsqueda Dicotómica.....	61

RESUMEN

En esta práctica se examina la aplicación de métodos de optimización del descenso de gradiente para determinar superficies mínimas de revolución, comparando enfoques de pasos fijos y variables. Se implementaron métodos de Fibonacci y Búsqueda dicotómica para la optimización de pasos, analizando su efectividad para encontrar soluciones óptimas. Partiendo de dos funciones iniciales (una línea recta y una parábola), examinamos el comportamiento de convergencia y la precisión de la solución en diferentes niveles de discretización. Con esta práctica se demuestra la implementación práctica de técnicas de optimización multivariable sin restricciones al tiempo que enfatiza la importancia de la selección adecuada de pasos en problemas de optimización. Los resultados indican la efectividad de los

métodos de paso variable para lograr mejores características de convergencia en comparación con los enfoques de paso fijo.

Palabras clave: Optimización sin restricciones, descenso del gradiente, método del trapecio, superficies de revolución, Fibonacci, búsqueda dicotómica.

INTRODUCCIÓN

El estudio de superficies mínimas de revolución mediante la optimización del descenso de gradientes representa una combinación útil y rigurosa entre el cálculo de variaciones y los métodos numéricos. En esta práctica se explora la implementación del descenso de gradiente con tasas de aprendizaje fijas y variables para encontrar la superficie de revolución óptima que minimice el área. El análisis abarca dos condiciones iniciales: una línea recta $u(x) = 1$ y una parábola $u(x) = x^2 - x + 1$, logrando una comparación integral de los comportamientos de convergencia. El tamaño de paso variable, determinado por Fibonacci y métodos dicotómicos, busca mantener un enfoque adaptativo para la optimización, mejorando potencialmente las tasas de convergencia y la precisión de la solución. En esta práctica examinamos sistemáticamente la relación entre el número de puntos de discretización (N) y la precisión de la solución final obtenida al tiempo que se incluye un estudio detallado de los perfiles de gradiente bajo tamaños de paso fijos y variables, proporcionando información sobre la dinámica de optimización y características de convergencia.

Descripción del laboratorio

Objetivos

La idea de este Laboratorio es estudiar un caso de optimización sin restricciones multivariables. Justamente, con la implementación del método del gradiente, se espera aproximar una curva que verifica ciertas condiciones. Esta actividad busca fomentar la exploración, comprensión e implementación del método del descenso del gradiente en un problema exigente desde el punto de vista computacional. Así, se espera que se logre determinar la conexión de este método con lo estudiado para la Actividad 1.

Planteamiento del Problema

El objetivo para esta entrega consiste en encontrar la curva que une los puntos $(0, 1)$ y $(1, 1)$, de forma que se minimice el área de la superficie de revolución alrededor del eje. Para ello, se quiere minimizar la integral:

$$I(u) = \int_0^1 u(x) \sqrt{1 + [u'(x)]^2} dx$$

Sujeta a las condiciones: $u(0) = 1 = u(1)$

En realidad, el área deseada es $2\pi I(u)$. Sin embargo, el factor 2π no es relevante y se puede normalizar para facilitar los cálculos.

Metodología

Para resolver este problema, realizaremos una discretización del intervalo $[0, 1]$, de forma que tomaremos intervalos de la forma: $\left[\frac{j}{n}, \frac{j+1}{n}\right]$, para $0 \leq j \leq n-1$.

Recordemos que si consideramos una función $f(x)$, la integración por trapecios nos da la fórmula:

$$\int_0^1 f(x) dx \approx \sum_{j=0}^{n-1} \frac{f(x_j) + f(x_{j+1})}{2} \cdot \frac{1}{n} = \sum_{j=0}^{n-1} T_j(f(x))$$

En nuestro caso, tendremos $f(x) = u(x) \sqrt{1 + [u'(x)]^2}$ y llamaremos $u_j = u(x_j)$. Como estamos aproximando por una poligonal, podemos considerar que:

$$u'(x_j) \approx \frac{u(x_{j+1}) - u(x_j)}{\frac{1}{n}} = n * [u(x_{j+1}) - u(x_j)]$$

Es decir, aproximamos la derivada por la derecha para el punto inicial y por la izquierda en el punto final. Por lo que la aproximación de la integral será:

Así pues, se obtiene la siguiente función, que podemos optimizar usando el método del gradiente:

$$\begin{array}{ccc} F & : & \mathbb{R}^{n+1} \rightarrow \mathbb{R} \\ & & u \mapsto T(u) \end{array}$$

Implementación

Para la implementación, se deben seguir los siguientes pasos:

- Calcular el gradiente de F
- Calcular la curva óptima para los distintos valores de n , teniendo en cuenta que:

$$T_j(u) = \frac{1}{2(n+1)} \cdot (u_{j+1} + u_j) \sqrt{1 + (n+1)^2 (u_{j+1} - u_j)^2}$$

- El gradiente está formado por:

$$\nabla F(u_0, u_1, \dots, u_n) = [g_0, g_1, \dots, g_n]^T$$

Donde:

$$g_k = \begin{cases} \frac{1}{2n} \left(\sqrt{1 + n^2(u_1 - u_0)^2} - \frac{n^2(u_1 - u_0)(u_0 + u_1)}{\sqrt{1 + n^2(u_1 - u_0)^2}} \right) & ; k = 0 \\ \frac{1}{2n} \left(\frac{n^2(u_k - u_{k-1})(u_{k-1} + u_k)}{\sqrt{1 + n^2(u_k - u_{k-1})^2}} + \sqrt{1 + n^2(u_k - u_{k-1})^2} \right. \\ \quad \left. + \sqrt{1 + n^2(u_{k+1} - u_k)^2} - \frac{n^2(u_{k+1} - u_k)(u_k + u_{k+1})}{\sqrt{1 + n^2(u_{k+1} - u_k)^2}} \right) & ; k = 1, \dots, n-1 \\ \frac{1}{2n} \left(\frac{n^2(u_n - u_{n-1})(u_{n-1} + u_n)}{\sqrt{1 + n^2(u_n - u_{n-1})^2}} + \sqrt{1 + n^2(u_n - u_{n-1})^2} \right) & ; k = n \end{cases}$$

Pautas de elaboración

- Deberás determinar cuál es la curva tal que, al considerar su sólido de revolución, el área de superficie es mínima. Para hacerlo, ten en cuenta que la incógnita del problema es una función (es necesario discretizarla, como se explica en los apartados anteriores). Al discretizar, el problema se convierte en uno multidimensional de dimensión alta.
- El informe debe entregarse en formato PDF, donde sea posible discriminar cada uno de los aspectos listados en la tabla anexa para la rúbrica.
- Deberás entregar un fichero en Matlab que permita verificar los resultados. Es importante destacar que ese fichero debe correr directamente sin señalar error alguno

Solución Exacta

```
clear all; close all;

% Valores iniciales asumidos
a0 = 1;
b0 = 0;

% Definimos sistema de ecuaciones para hacer una resolución numérica (más eficiente)
fun = @(x) [x(1)*cosh(-x(2)/x(1)) - 1;
            x(1)*cosh((1-x(2))/x(1)) - 1];

% Usamos fsolve para solución numérica
options = optimoptions('fsolve', 'Display', 'off');
sol = fsolve(fun, [a0; b0], options);

% Soluciones de a y b
a_val = sol(1);
b_val = sol(2);
fprintf('a = %f\nb = %f\n', a_val, b_val);
```

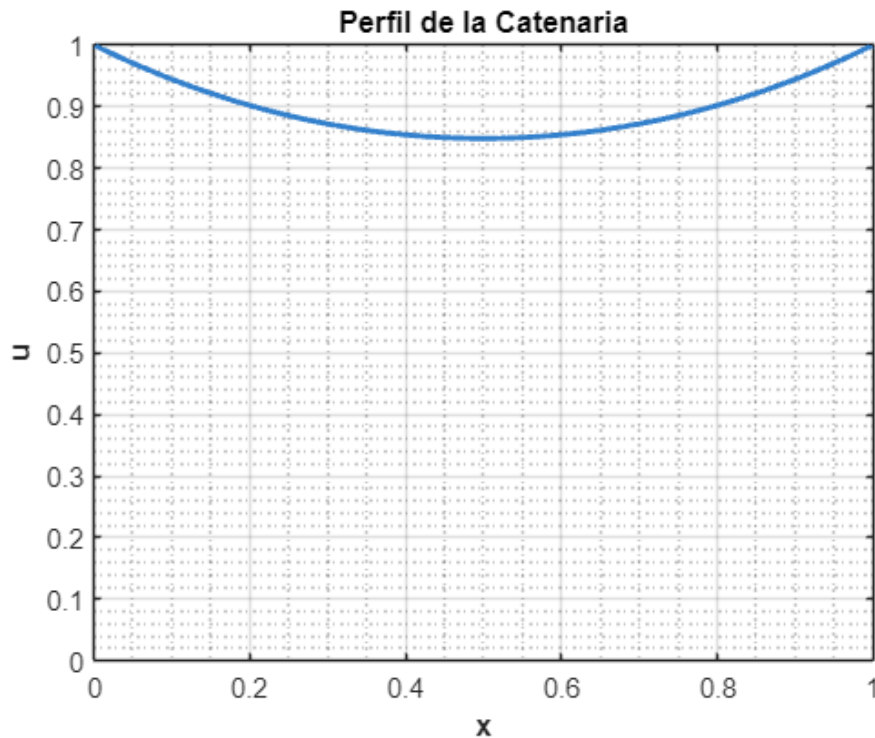
```
a = 0.848338
b = 0.500000
```

```
% Función para graficar
u_func = @(x) a_val * cosh((x - b_val)/a_val);
```

```

% 2D Plot
figure(1)
x_range = 0:0.01:1;
plot(x_range, u_func(x_range), 'LineWidth', 2, 'Color', [0.2 0.5 0.8])
grid on
xlabel('x', 'FontSize', 12, 'FontWeight', 'bold')
ylabel('u', 'FontSize', 12, 'FontWeight', 'bold')
title('Perfil de la Catenaria', 'FontSize', 14, 'FontWeight', 'bold')
axis([0 1 0 max(u_func(x_range))])
set(gca, 'FontSize', 10)
box on
grid minor

```



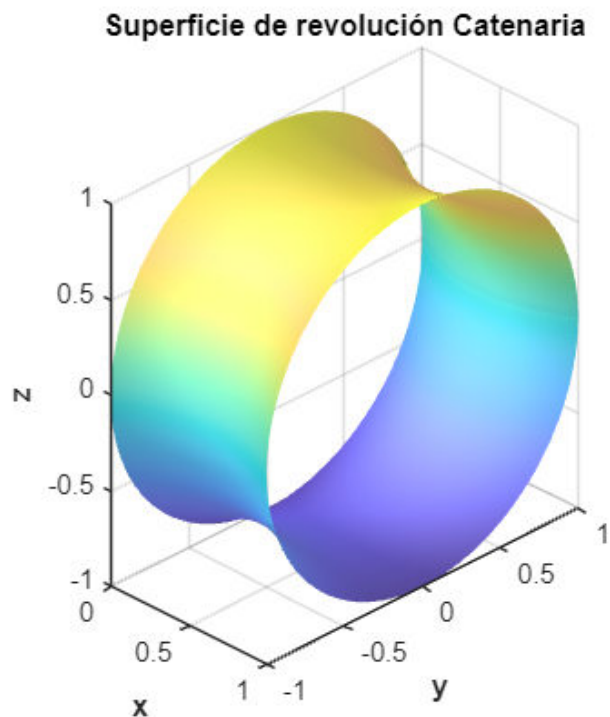
```

% 3D Plot Superficie de revolución
figure(2)
[X, THETA] = meshgrid(x_range, 0:pi/50:2*pi);
R = u_func(X);
Y = R .* cos(THETA);
Z = R .* sin(THETA);

surf(X, Y, Z, 'EdgeColor', 'none', 'FaceAlpha', 0.8)
xlabel('x', 'FontSize', 12, 'FontWeight', 'bold')
ylabel('y', 'FontSize', 12, 'FontWeight', 'bold')
zlabel('z', 'FontSize', 12, 'FontWeight', 'bold')
title('Superficie de revolución Catenaria', 'FontSize', 14, 'FontWeight',
'bold')
axis equal
colormap('parula') % Parula para mejor contraste
shading interp
lighting gouraud

```

```
camlight('headlight')
material([0.7 0.5 0.3 1])
grid on
set(gca, 'FontSize', 10)
view(45, 30)
```



```
% Calculo del área
% Derivada de u
du_dx = @(x) a_val * sinh((x - b_val)/a_val) * (1/a_val);
% Integrando de la función
integrando = @(x) 2*pi * u_func(x) .* sqrt(1 + du_dx(x).^2);
% Integración numérica
A = integral(integrando, 0, 1);

fprintf('Area de Revolución Exacta = %f\n', A);
```

Area de Revolución Exacta = 5.991797

Método de paso fijo

Partiendo de recta $u(x) = 1$

```
clc; clear; close all;
format longg;
%% parámetros...
n_vector = [5, 10, 15, 20, 25, 50];
t_vector = [0.001, 0.01, 0.15];

delta_1 = 1e-5;
```

```

delta_2 = 0.75;
max_k = 10000;

% Estructura para guardar resultados
resultados = struct();

% % Crear carpeta para guardar resultados
output_folder = 'resultados_pdf';
if ~exist(output_folder, 'dir')
    mkdir(output_folder);
end

% % salidas...
l = length(n_vector);
tiledlayout(1, l);
iter=0;
tic
for n_idx = 1:length(n_vector)
    for t_idx = 1:length(t_vector)
        % Definir parámetros actuales
        n = n_vector(n_idx);
        t = t_vector(t_idx);
        iter=iter+1;
        [tabla_iteraciones, u_min_aprox, u_hist, k, texec] =
metodo_gradiente_paso_fijo(n, t, delta_1, delta_2, max_k);
        % calculo valores exactos...
        x = 0:1/n:1;
        a = 0.848338;
        u_min_exacta = a*cosh((x-1/2)/a); % catenaria exacta
        I_min_exacta = (a/2)*(1 + a*sinh(1/a)); % se ha obviado el 2*pi
        fprintf('*Solución óptima encontrada en la iteración %d, para n = %d,
t = %f\n\n', k, n, t);
        % tabulo...
        disp(array2table(tabla_iteraciones(end-3:end,:), 'VariableNames',
{'k', 'F(u)', '||u_{k+1} - u_k||', '||grad_F(u_{k+1})||'}));
        F_min_aprox = F(u_min_aprox);

        % Guardar resultados en la estructura
        resultados(iter).n = n;
        resultados(iter).t = t;
        resultados(iter).tabla_iteraciones = tabla_iteraciones;
        resultados(iter).u_min_aprox = u_min_aprox;
        resultados(iter).u_hist = u_hist;
        resultados(iter).k = k;
        resultados(iter).texec = texec;
        resultados(iter).u_min_exacta = u_min_exacta;
        resultados(iter).I_min_exacta = I_min_exacta;
        resultados(iter).F_min_aprox = F_min_aprox;
        resultados(iter).error = abs(F_min_aprox - I_min_exacta);

        fprintf('Valor mínimo de F(u) = F_min_aprox = %f\n|F_min_aprox -
I_min_exacta| = %f\n\n', F_min_aprox, abs(F_min_aprox - I_min_exacta));
    end
end

```

```

% grafico...
figure(iter);
plot(x, u_hist(1:20:end,:), 'r', ...           % Color rojo para
u_hist                                         % Color rojo para
x, u_min_aprox, 'r', ...                       % Color rojo para
u_min_aprox                                   % Color rojo para
x, u_min_exacta, '-b', 'MarkerSize', 10); % Color azul para
u_min_exacta con estilo y tamaño del marcador
xlabel('$x$', 'Interpreter', 'latex');
ylabel('$u(x)$', 'Interpreter', 'latex');
legend('$u_{k}(x)$', '$u_{\min}(x)$', 'Interpreter', 'latex');
title(['iter = ', num2str(k), ', texec = ', num2str(texec), ' s, n = ',
num2str(n), ', t = ', num2str(t)]);
grid on;

% Guardar la figura
% saveas(gcf, fullfile(output_folder, sprintf('final_n%d_t%.3f.pdf',
n, t)));
end
end

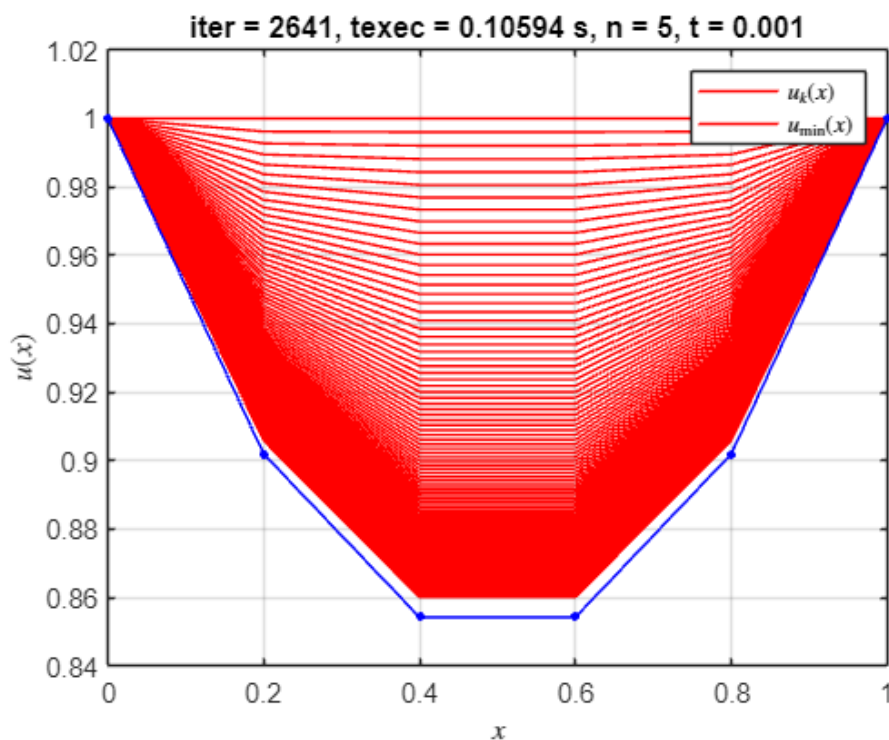
```

*Solución óptima encontrada en la iteración 2641, para $n = 5$, $t = 0.001000$

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
2638	0.955626522738047	1.00272364775583e-05	0.730368462705179
2639	0.955626422252063	1.00153713191927e-05	0.730387682354737
2640	0.955626322003745	1.00035205838826e-05	0.73040687906178
2641	0.955626221992524	9.99168425298791e-06	0.730426052854624

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955626$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.002002$

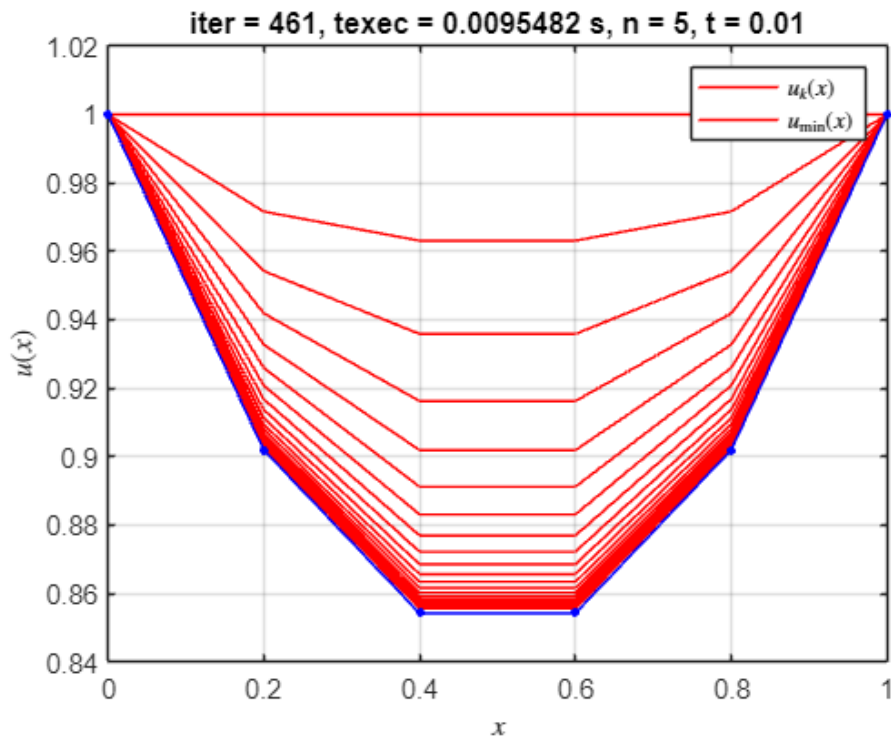


*Solución óptima encontrada en la iteración 461, para $n = 5$, $t = 0.010000$

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
---	------	---------------------	------------------------------

458	0.955584097673837	1.02413328827885e-05	0.745090974855062
459	0.955584087245861	1.01231594532156e-05	0.745110234093893
460	0.955584077057147	1.00063535458431e-05	0.745129270880659
461	0.9555840671022	9.89089924541878e-06	0.745148087794666

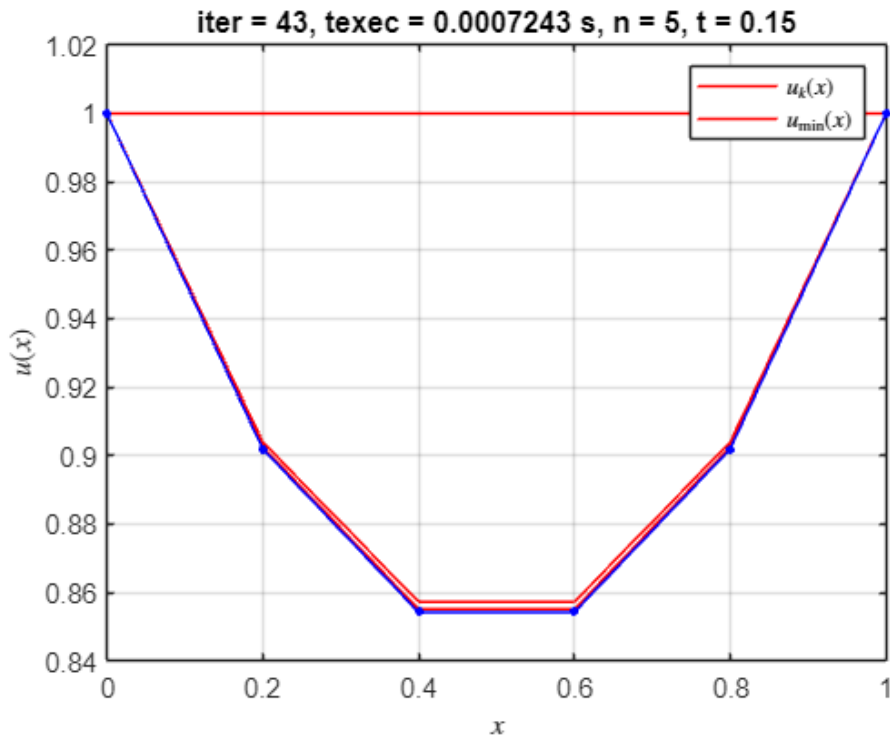
Valor mínimo de $F(u) = F_{\min_aprox} = 0.955584$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.001960$



*Solución óptima encontrada en la iteración 43, para $n = 5$, $t = 0.150000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
40	0.955583648707284	1.75629487232882e-05	0.746601625392338
41	0.955583646828401	1.45310502991858e-05	0.746629244082111
42	0.955583645542223	1.20226831291745e-05	0.746652094828346
43	0.955583644661759	9.94740466308303e-06	0.746671000972261

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955584$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.001959$

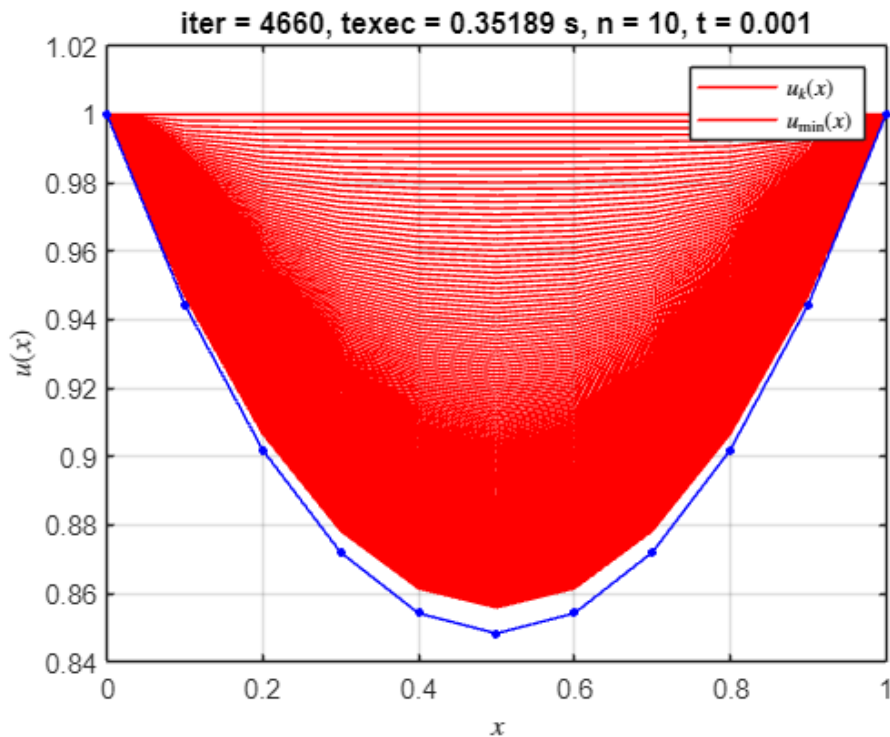


*Solución óptima encontrada en la iteración 4660, para n = 10, t = 0.001000

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
4657	0.954198901405223	1.00162670725477e-05	0.724480347379062
4658	0.954198801109982	1.00102040923685e-05	0.724494598743755
4659	0.954198700936122	1.00041449223905e-05	0.724508841321767
4660	0.954198600883496	9.99808956012424e-06	0.724523075118804

Valor mínimo de $F(u) = F_{\min_aprox} = 0.954199$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000574$

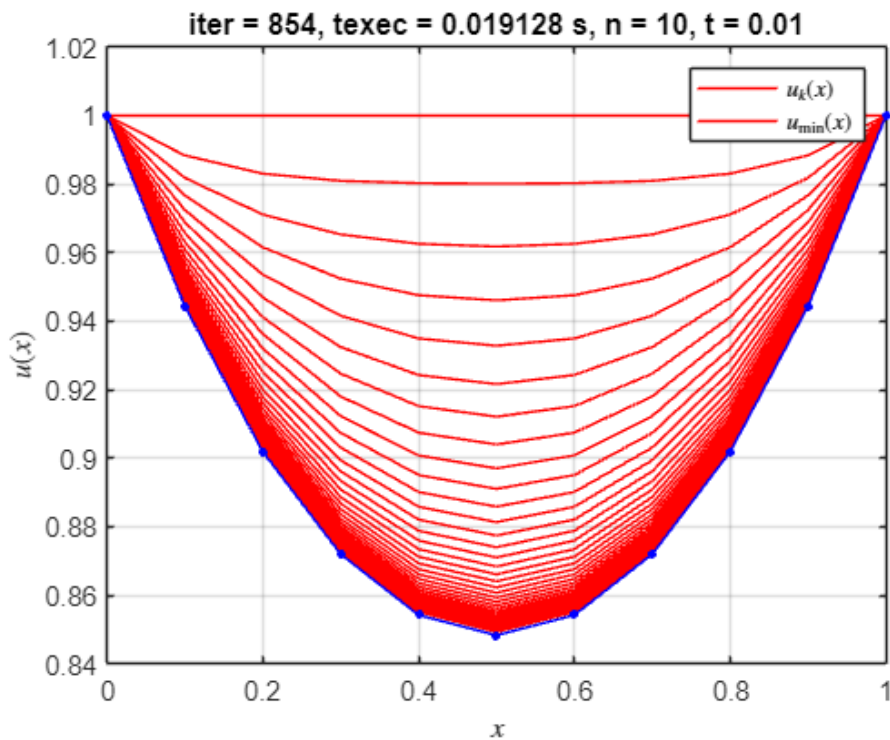


*Solución óptima encontrada en la iteración 854, para n = 10, t = 0.010000

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
---	------	---------------------	------------------------------

851	0.954115829630747	1.01686421001337e-05	0.745835988394174
852	0.954115819320819	1.01092431141167e-05	0.745850127269322
853	0.954115809130987	1.00501925344316e-05	0.745864183382601
854	0.95411579905985	9.99148830077652e-06	0.745878157221437

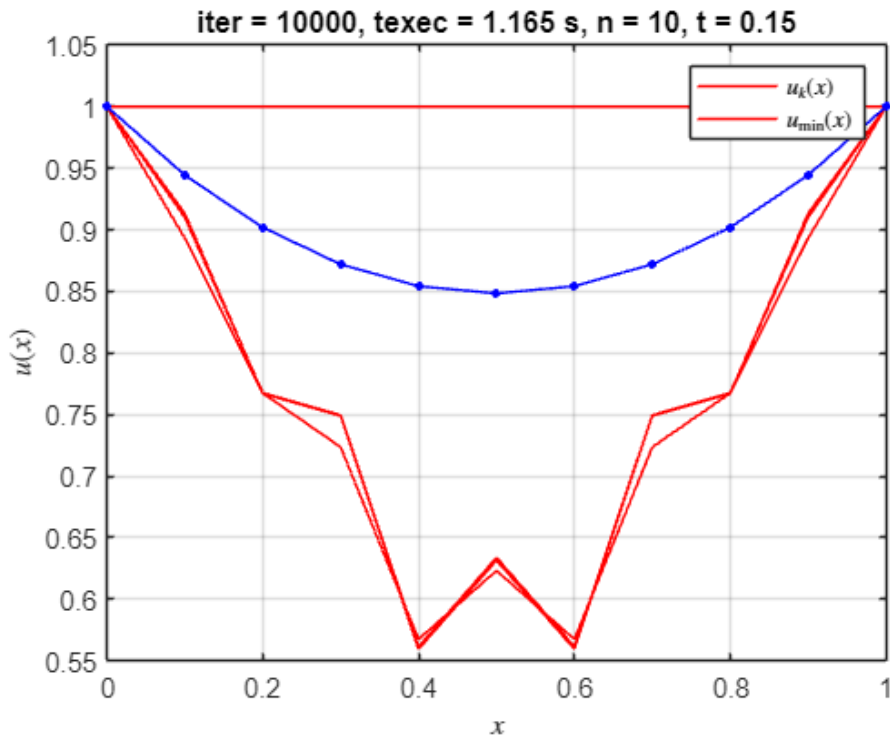
Valor mínimo de $F(u) = F_{\min_aprox} = 0.954116$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000492$



*Solución óptima encontrada en la iteración 10000, para $n = 10$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	1.12904143822076	0.258179589445983	2.06033133545535
9998	1.14287899353169	0.258179589445983	1.98910249814077
9999	1.12904143822076	0.258179589445983	2.06033133545535
10000	1.14287899353169	0.258179589445983	1.98910249814077

Valor mínimo de $F(u) = F_{\min_aprox} = 1.129041$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.175417$

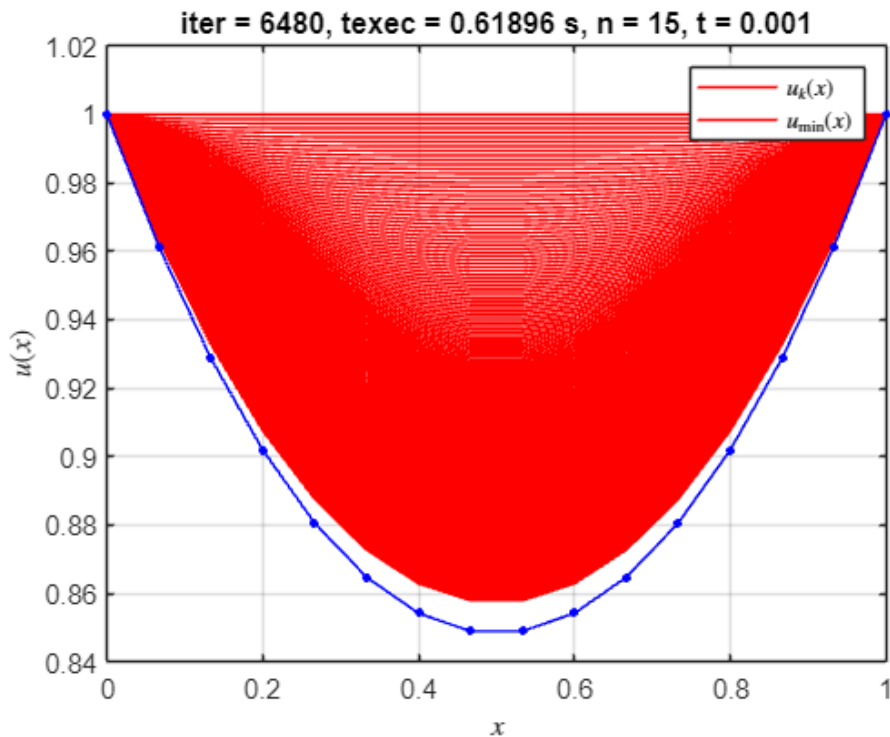


*Solución óptima encontrada en la iteración 6480, para n = 15, t = 0.001000

k	F(u)	$ u_{k+1} - u_k $	$ grad_F(u_{k+1}) $
---	------	---------------------	------------------------

6477	0.953967127460957	1.00104140776443e-05	0.719314890241955
6478	0.953967027273005	1.00063308805193e-05	0.71932668193612
6479	0.953966927166767	1.00022494255731e-05	0.719338468701134
6480	0.953966827142177	9.99816971214557e-06	0.719350250539196

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953967$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000343$



*Solución óptima encontrada en la iteración 1228, para n = 15, t = 0.010000

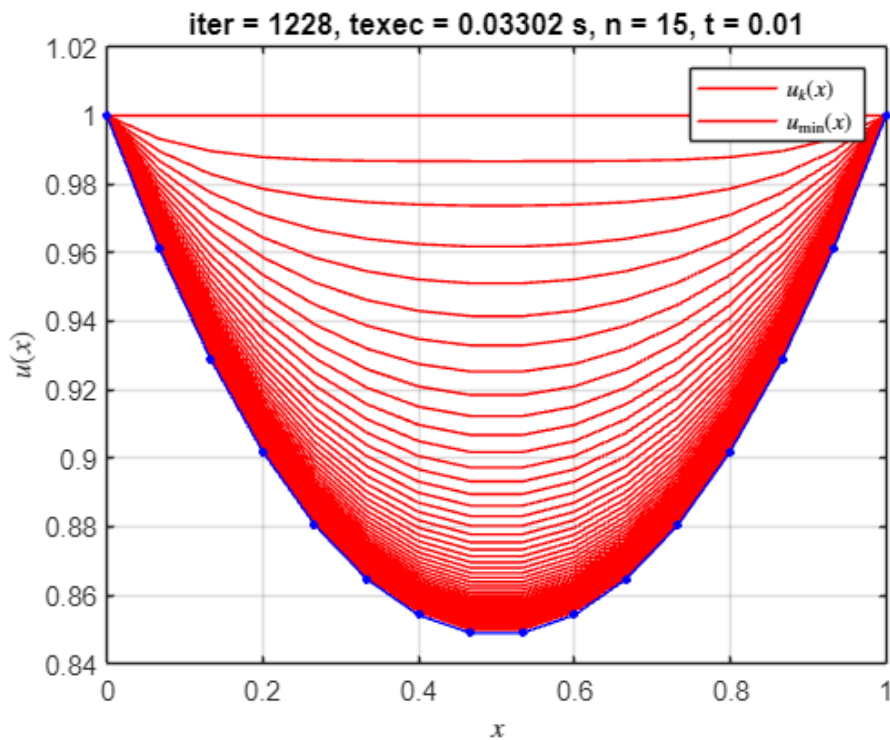
k	F(u)	$ u_{k+1} - u_k $	$ grad_F(u_{k+1}) $
---	------	---------------------	------------------------

--	--	--	--

1225	0.953843642657957	1.00978897603945e-05	0.745571830531366
1226	0.953843632481134	1.00584463461272e-05	0.745583405455142
1227	0.953843622383659	1.00191577789337e-05	0.745594935040469
1228	0.953843612364911	9.98002344482953e-06	0.74560641946631

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953844$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000219$

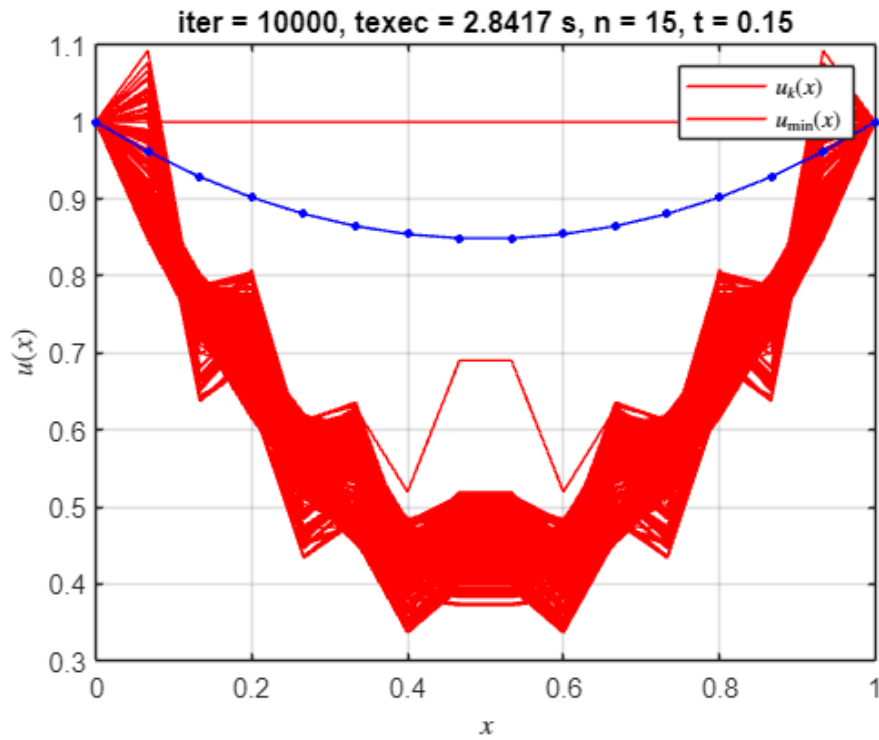


*Solución óptima encontrada en la iteración 10000, para $n = 15$, $t = 0.150000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	1.11848525255849	0.177995899920808	1.81360763357685
9998	1.11759631706904	0.188284865446705	1.81349540781208
9999	1.12861342365255	0.188327170099119	1.76496093123722
10000	1.11571159795852	0.177685491931081	1.69472192681556

Valor mínimo de $F(u) = F_{\min_aprox} = 1.115066$

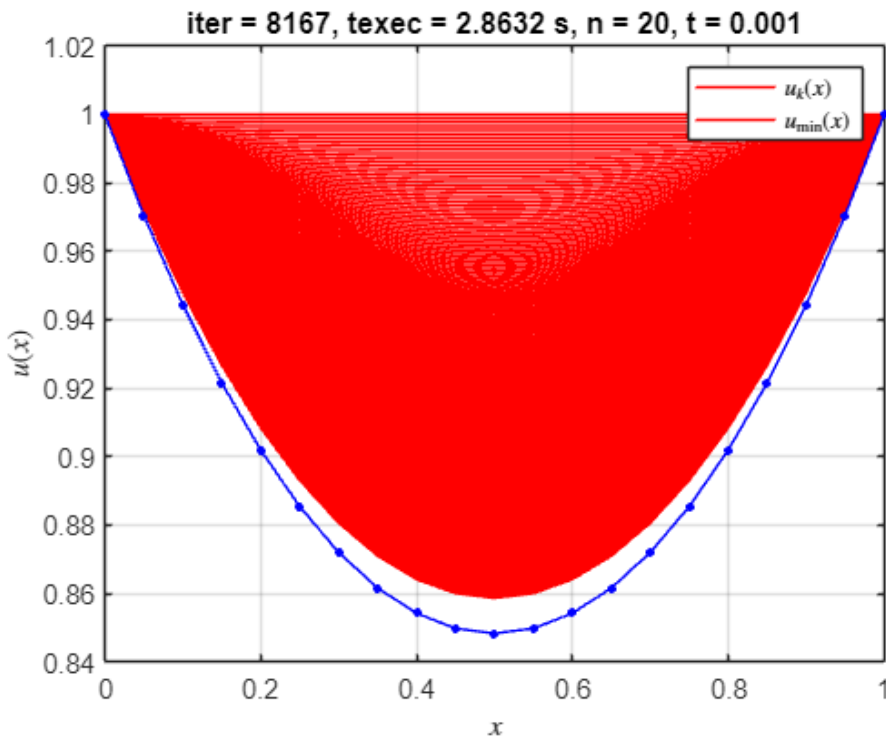
$|F_{\min_aprox} - I_{\min_exacta}| = 0.161442$



*Solución óptima encontrada en la iteración 8167, para $n = 20$, $t = 0.001000$

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
8164	0.953912270850164	1.00084023782208e-05	0.714870940009798
8165	0.953912170697489	1.00053164235918e-05	0.714881229977036
8166	0.953912070606565	1.00022314702868e-05	0.714891516677354
8167	0.953911970577354	9.99914751814148e-06	0.71490180011187

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953912$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000288$



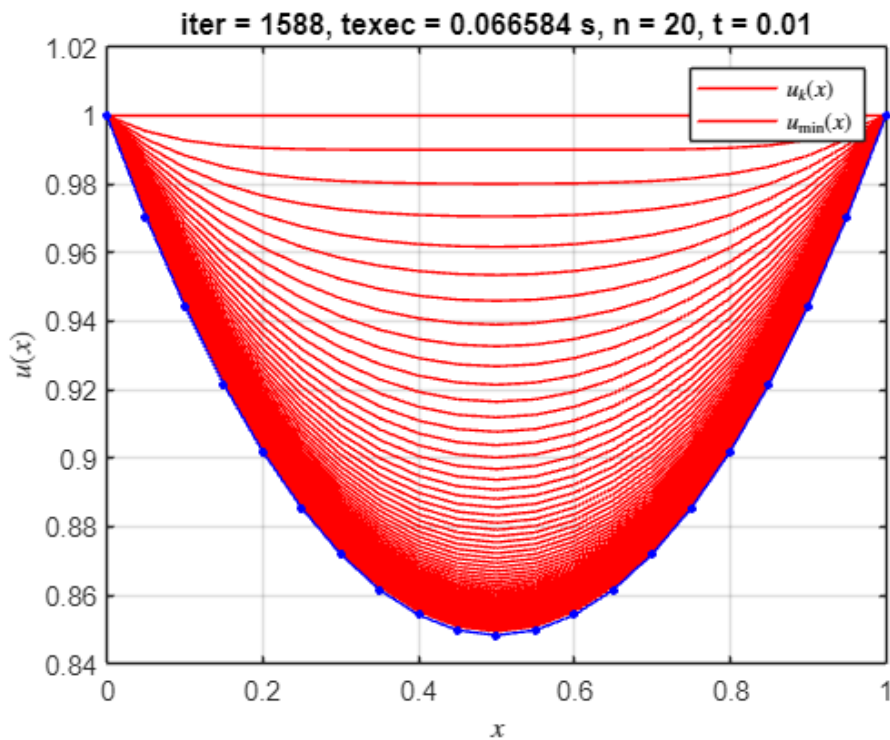
*Solución óptima encontrada en la iteración 1588, para $n = 20$, $t = 0.010000$

k	F(u)	$ u_{k+1} - u_k $	$ \text{grad}_F(u_{k+1}) $
---	------	---------------------	------------------------------

1585	0.953748605378644	1.00645690299098e-05	0.74521343043422
1586	0.95374859526395	1.00350393029291e-05	0.745223461377235
1587	0.953748585208521	1.00055967200395e-05	0.745233462790861
1588	0.953748575212011	9.97624102127732e-06	0.745243434762815

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953749$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000124$

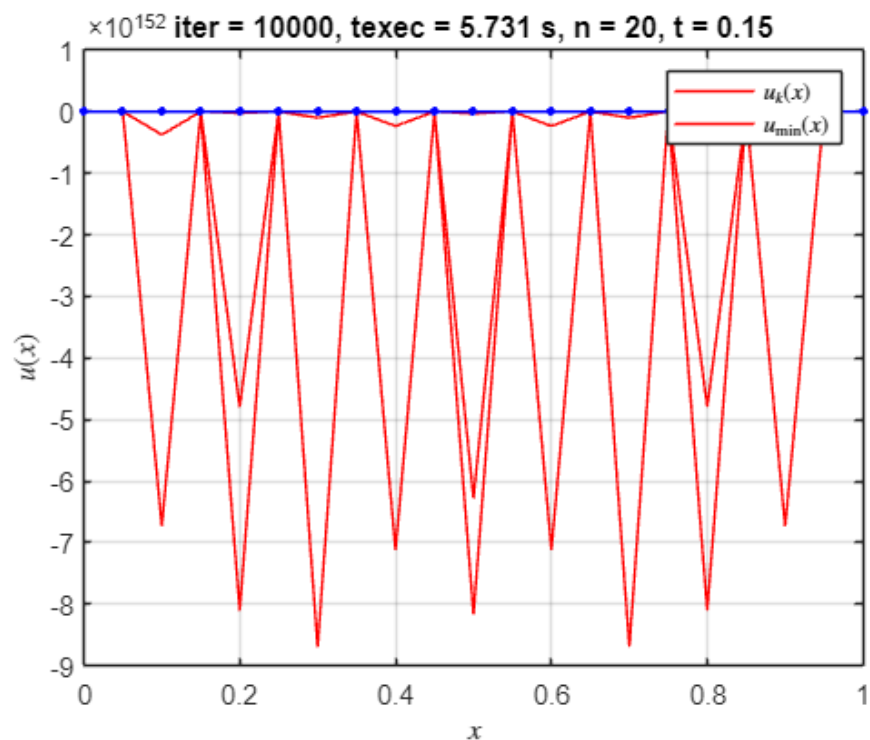


*Solución óptima encontrada en la iteración 10000, para $n = 20$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	-Inf	0.152079992293084	1.36826208292915
9998	-Inf	0.152079992293084	1.51607440516857
9999	-Inf	0.152079992293084	1.36826208292915
10000	-Inf	0.152079992293084	1.51607440516857

Valor mínimo de $F(u) = F_{\min_aprox} = -Inf$

$|F_{\min_aprox} - I_{\min_exacta}| = Inf$

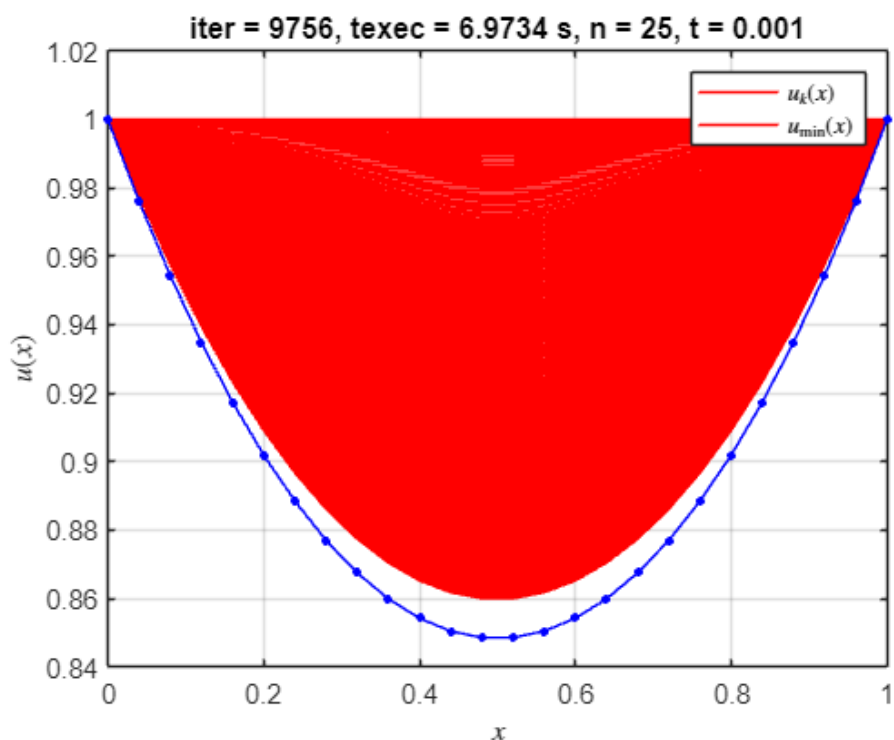


*Solución óptima encontrada en la iteración 9756, para n = 25, t = 0.001000

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9753	0.953908372566329	1.00056167223021e-05	0.710943523548078
9754	0.953908272466391	1.0003132517474e-05	0.710952776735386
9755	0.953908172416152	1.00006489652488e-05	0.710962027547844
9756	0.953908072415588	9.99816606530498e-06	0.710971275986107

Valor mínimo de F(u) = F_min_aprox = 0.953908

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000284$



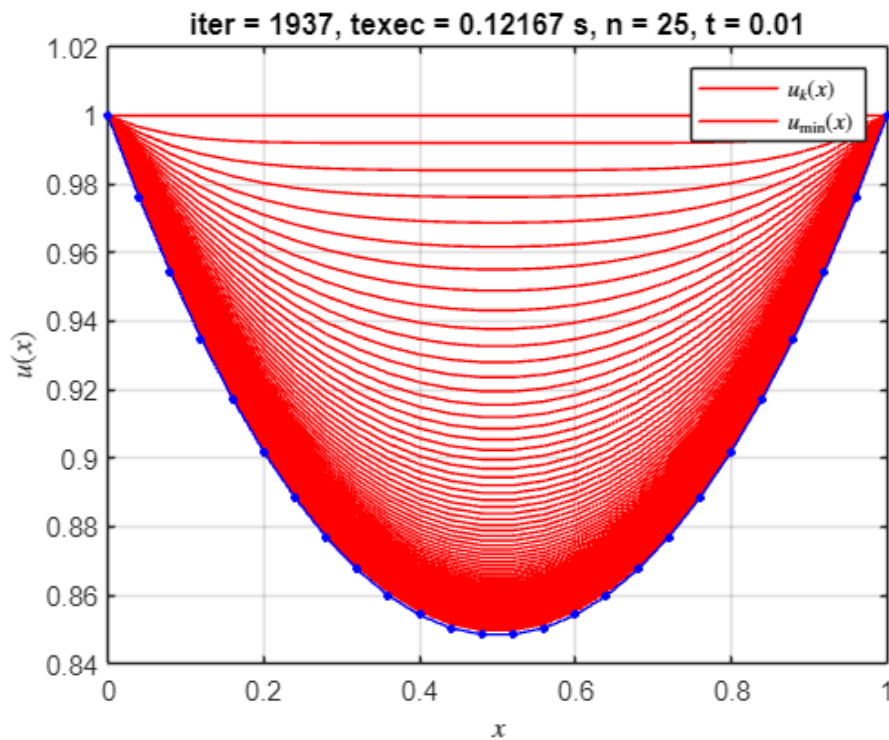
*Solución óptima encontrada en la iteración 1937, para n = 25, t = 0.010000

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------

1934	0.953704839956438	1.00556821657521e-05	0.744853310441506
1935	0.953704829856643	1.00320548728349e-05	0.744862294234151
1936	0.953704819804254	1.00084834559864e-05	0.744871256837296
1937	0.953704809799047	9.9849677812648e-06	0.744880198301431

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953705$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000081$

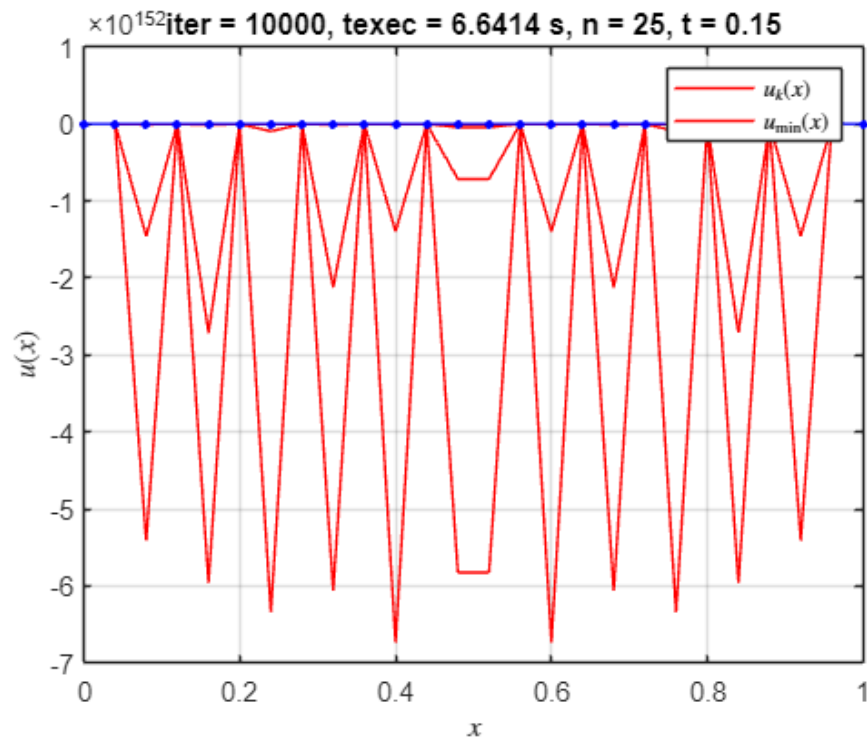


*Solución óptima encontrada en la iteración 10000, para $n = 25$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	-Inf	0.170947455651523	1.55189195043942
9998	-Inf	0.170947455651523	1.6985605131146
9999	-Inf	0.170947455651523	1.55189195043942
10000	-Inf	0.170947455651523	1.6985605131146

Valor mínimo de $F(u) = F_{\min_aprox} = -Inf$

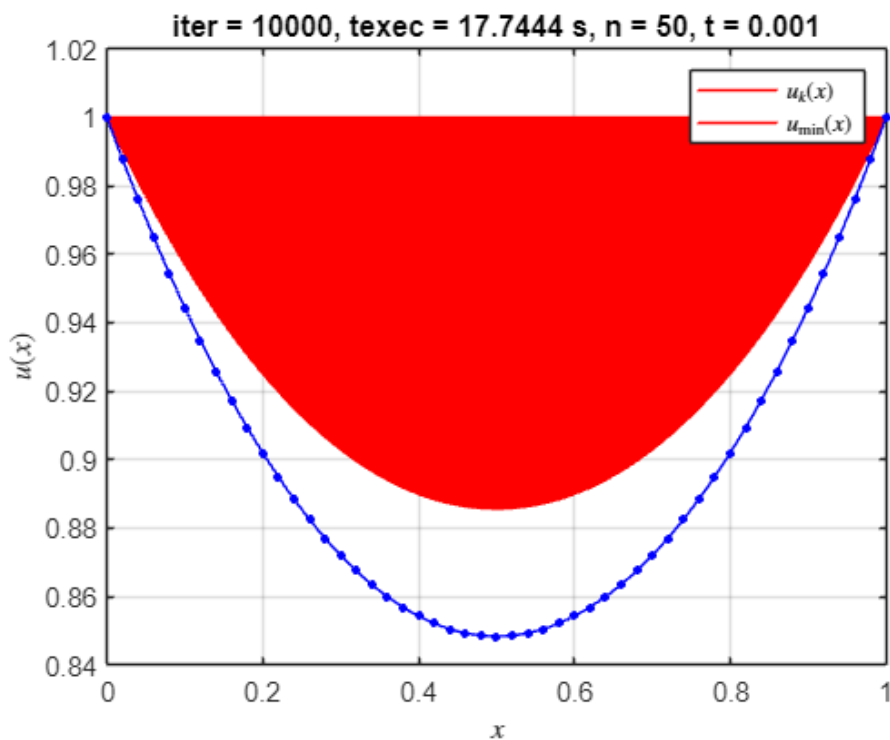
$|F_{\min_aprox} - I_{\min_exacta}| = Inf$



*Solución óptima encontrada en la iteración 10000, para $n = 50$, $t = 0.001000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
9997	0.95590701592008	2.45283423633017e-05	0.620993828914641
9998	0.955906414322832	2.45248908709687e-05	0.621011196827833
9999	0.955905812894876	2.45214399386799e-05	0.621028562076154
10000	0.955905211636164	2.45179895663597e-05	0.621045924660082

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955905$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.002280$



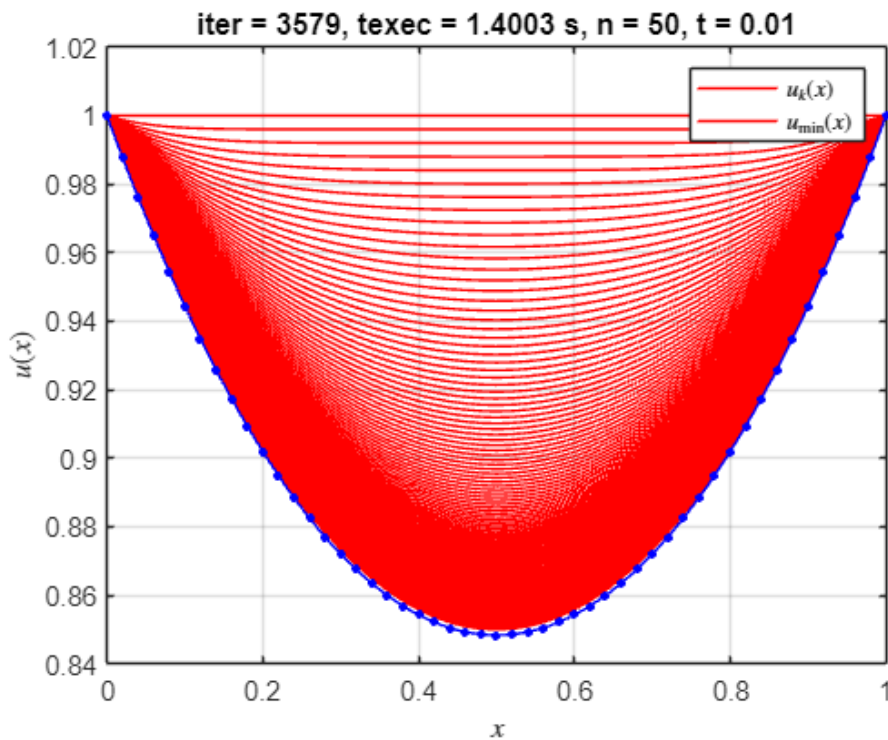
*Solución óptima encontrada en la iteración 3579, para $n = 50$, $t = 0.010000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------------

3576	0.953648032738506	1.00289305314951e-05	0.743332713489655
3577	0.953648022686488	1.00171113285069e-05	0.743339077778477
3578	0.953648012658148	1.00053061815737e-05	0.743345434524547
3579	0.953648002653432	9.99351507376945e-06	0.743351783736934

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953648$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.000024$

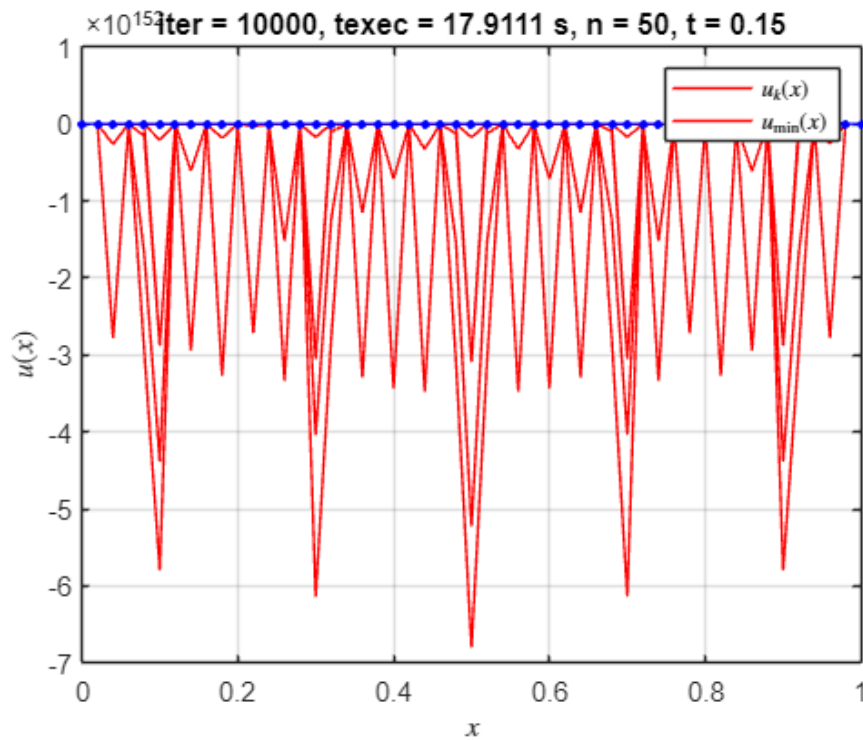


*Solución óptima encontrada en la iteración 10000, para $n = 50$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	-Inf	0.186786338656144	1.86587795664023
9998	-Inf	0.186786338656144	1.72370729187847
9999	-Inf	0.186786338656144	1.86587795664023
10000	-Inf	0.186786338656144	1.72370729187847

Valor mínimo de $F(u) = F_{\min_aprox} = -Inf$

$|F_{\min_aprox} - I_{\min_exacta}| = Inf$



```
elapsedTime = toc;
% Muestra el tiempo transcurrido
fprintf('El algoritmo tardó %.4f segundos en ejecutarse.\n', elapsedTime);
```

El algoritmo tardó 18.5831 segundos en ejecutarse.

```
save('resultados_t_fijo.mat','resultados');
```

Partiendo de parábola $u(x) = x^2 - x + 1$;

```
% Insertar código para aproximación de parábola
clc; clear; close all;
format longg;
% % parámetros...
n_vector = [5, 10, 15, 20, 25, 50];
t_vector = [0.001, 0.01, 0.15];

delta_1 = 1e-5;
delta_2 = 0.75;
max_k = 10000;

% Estructura para guardar resultados
resultados = struct();

% % Crear carpeta para guardar resultados
output_folder = 'resultados_pdf';
if ~exist(output_folder, 'dir')
    mkdir(output_folder);
end
```

```

%% % salidas...
l = length(n_vector);
tiledlayout(1, l);
iter=0;
tic
for n_idx = 1:length(n_vector)
    for t_idx = 1:length(t_vector)
        % Definir parámetros actuales
        n = n_vector(n_idx);
        t = t_vector(t_idx);
        iter=iter+1;
        [tabla_iteraciones, u_min_aprox, u_hist, k, texec] =
metodo_gradiente_paso_fijo_para(n, t, delta_1, delta_2, max_k);
        % calculo valores exactos...
        x = 0:1/n:1;
        a = 0.848338;
        u_min_exacta = a*cosh((x-1/2)/a); % catenaria exacta
        I_min_exacta = (a/2)*(1 + a*sinh(1/a)); % se ha obviado el 2*pi
        fprintf('* Parabola: Solución óptima encontrada en la iteración %d,
para n = %d, t = %f\n\n', k, n, t);
        % tabulo...
        disp(array2table(tabla_iteraciones(end-3:end,:), 'VariableNames',
{'k', 'F(u)', '||u_{k+1} - u_k||', '||grad_F(u_{k+1})||'}));
        F_min_aprox = F(u_min_aprox);

        % Guardar resultados en la estructura
        resultados(iter).n = n;
        resultados(iter).t = t;
        resultados(iter).tabla_iteraciones = tabla_iteraciones;
        resultados(iter).u_min_aprox = u_min_aprox;
        resultados(iter).u_hist = u_hist;
        resultados(iter).k = k;
        resultados(iter).texec = texec;
        resultados(iter).u_min_exacta = u_min_exacta;
        resultados(iter).I_min_exacta = I_min_exacta;
        resultados(iter).F_min_aprox = F_min_aprox;
        resultados(iter).error = abs(F_min_aprox - I_min_exacta);

        fprintf('Valor mínimo de F(u) = F_min_aprox = %f\n|F_min_aprox -
I_min_exacta| = %f\n\n', F_min_aprox, abs(F_min_aprox - I_min_exacta));
        % grafico...
        figure(iter);
        plot(x, u_hist(1:20:end,:), 'r', ... % Color rojo para
u_hist
            x, u_min_aprox, 'r', ... % Color rojo para
u_min_aprox
            x, u_min_exacta, '.-b', 'MarkerSize', 10); % Color azul para
u_min_exacta con estilo y tamaño del marcador
        xlabel('$x$', 'Interpreter', 'latex');
        ylabel('$u(x)$', 'Interpreter', 'latex');
        legend('$u_{k}(x)$', '$u_{\min}(x)$', 'Interpreter', 'latex');

```

```

        title(['iter = ', num2str(k), ', texec = ', num2str(texec), ' s, n = ', num2str(n), ', t = ', num2str(t)]);
        grid on;

        % Guardar la figura
        % saveas(gcf, fullfile(output_folder, sprintf('final_n%d_t%.3f.pdf',
n, t)));
    end
end

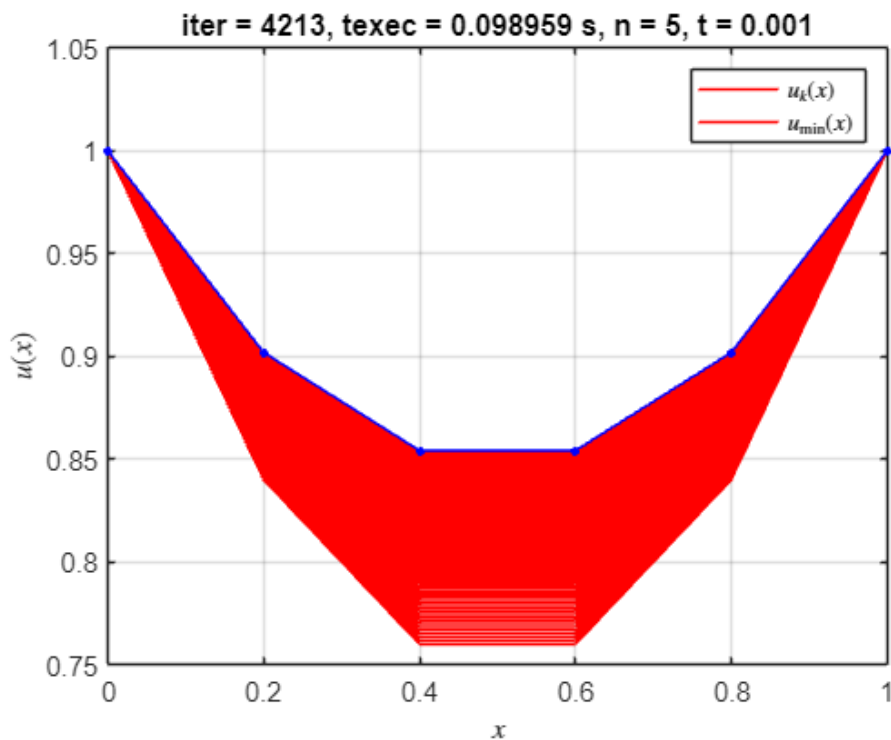
```

* Parabola: Solución óptima encontrada en la iteración 4213, para $n = 5$, $t = 0.001000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
4210	0.955585326583892	1.96465742220092e-06	0.750009077119861
4211	0.955585322726221	1.96240988811541e-06	0.750005355421149
4212	0.955585318877371	1.96016491048943e-06	0.750001637970934
4213	0.955585315037322	1.95792248645072e-06	0.749997924764405

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955585$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.001961$

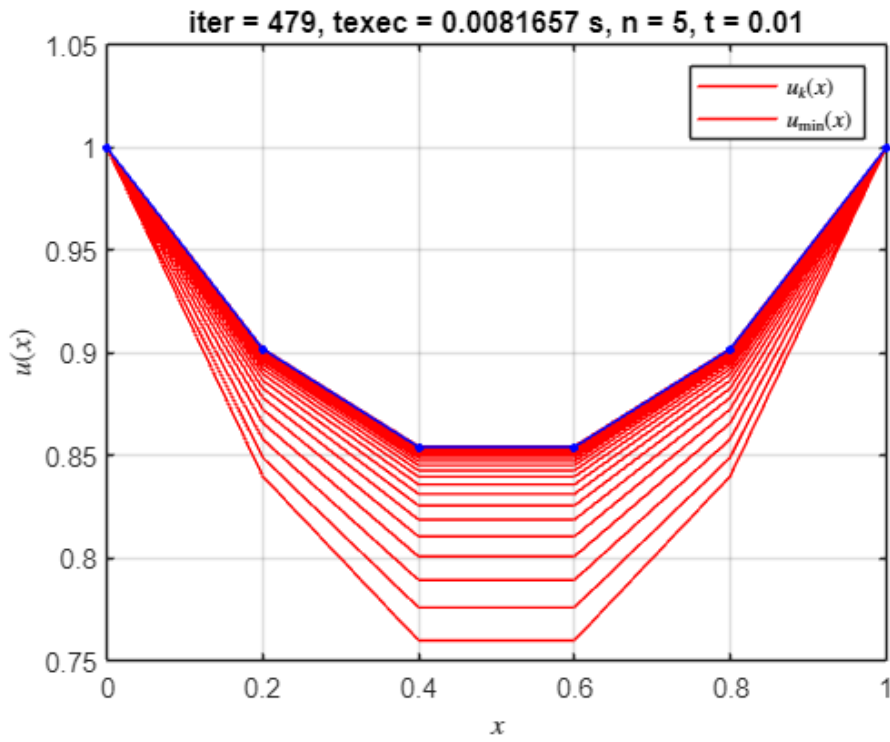


* Parabola: Solución óptima encontrada en la iteración 479, para $n = 5$, $t = 0.010000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
476	0.955584104989456	1.03029861030462e-05	0.748445585060375
477	0.955584094435187	1.01847993907761e-05	0.748426249831057
478	0.95558408412167	1.00679644300026e-05	0.748407136167501
479	0.955584074043421	9.95246585109498e-06	0.748388241540872

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955584$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.001960$

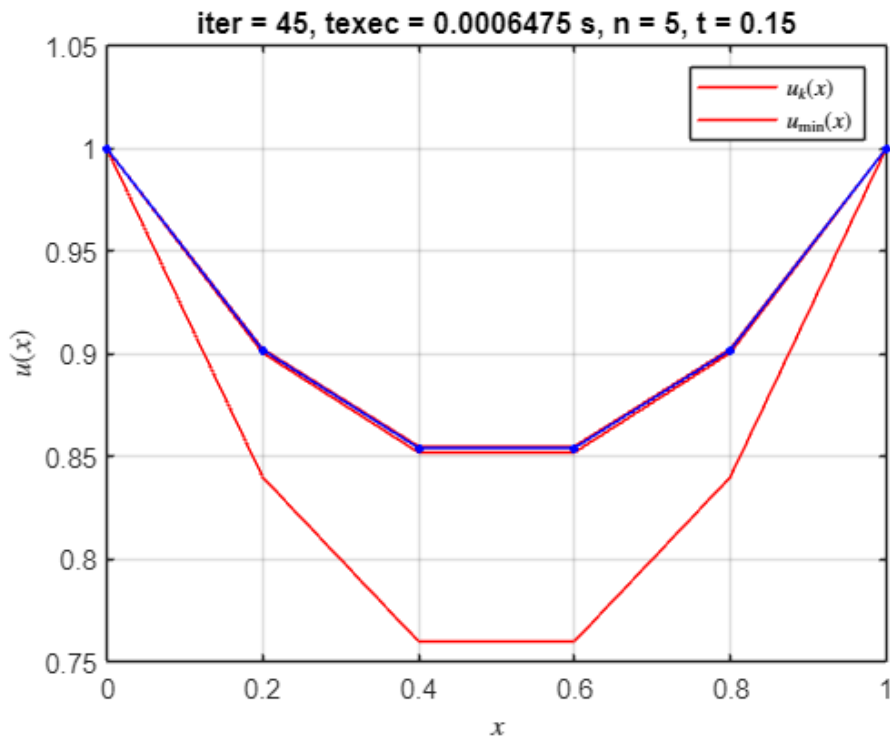


* Parábola: Solución óptima encontrada en la iteración 45, para $n = 5$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
42	0.955583648310193	1.69636837514454e-05	0.746916227324254
43	0.955583646557238	1.40370119183676e-05	0.746889552574978
44	0.955583645356972	1.16151417047391e-05	0.746867479796112
45	0.955583644535154	9.61104260942306e-06	0.746849215259983

Valor mínimo de $F(u) = F_{\min_aprox} = 0.955584$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.001959$

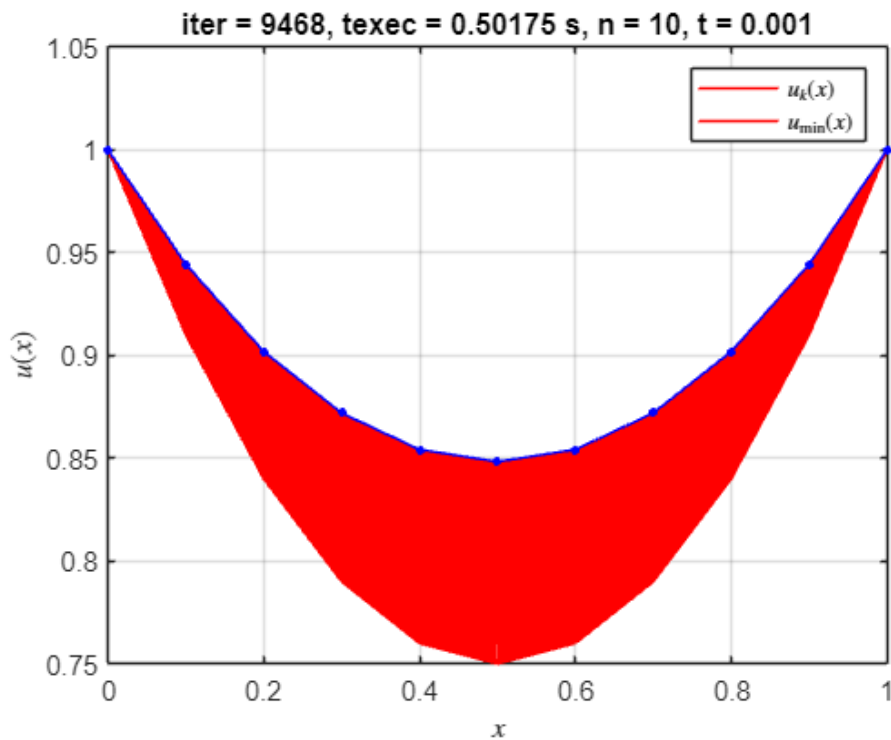


* Parábola: Solución óptima encontrada en la iteración 9468, para $n = 10$, $t = 0.001000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------

9465	0.954115398067448	7.26623936296902e-07	0.750002555628375
9466	0.954115397539619	7.26202525838954e-07	0.750001543674738
9467	0.954115397012401	7.25781359098963e-07	0.750000532307083
9468	0.954115396485795	7.25360435970809e-07	0.749999521525068

Valor mínimo de $F(u) = F_{\min_aprox} = 0.954115$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000491$

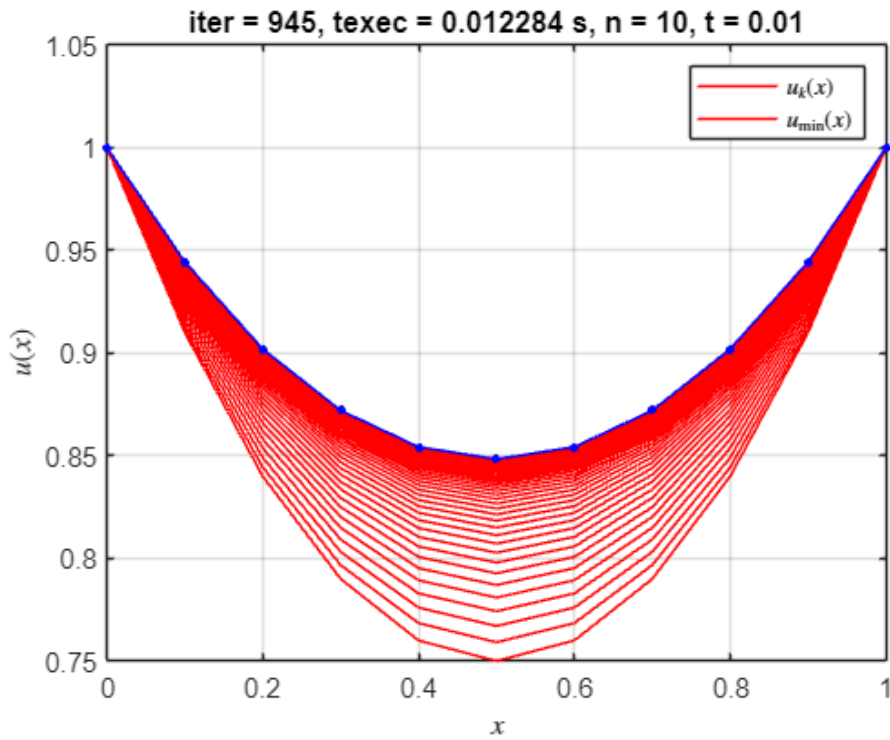


* Parábola: Solución óptima encontrada en la iteración 945, para $n = 10$, $t = 0.010000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
---	--------	-------------------------	---------------------------

942	0.954115415302825	7.4025218995446e-06	0.750025998448865
943	0.954115409838982	7.3595925644851e-06	0.750015743128982
944	0.954115404438327	7.31691142930183e-06	0.750005547190848
945	0.954115399100132	7.27447706774072e-06	0.74999541029221

Valor mínimo de $F(u) = F_{\min_aprox} = 0.954115$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000491$

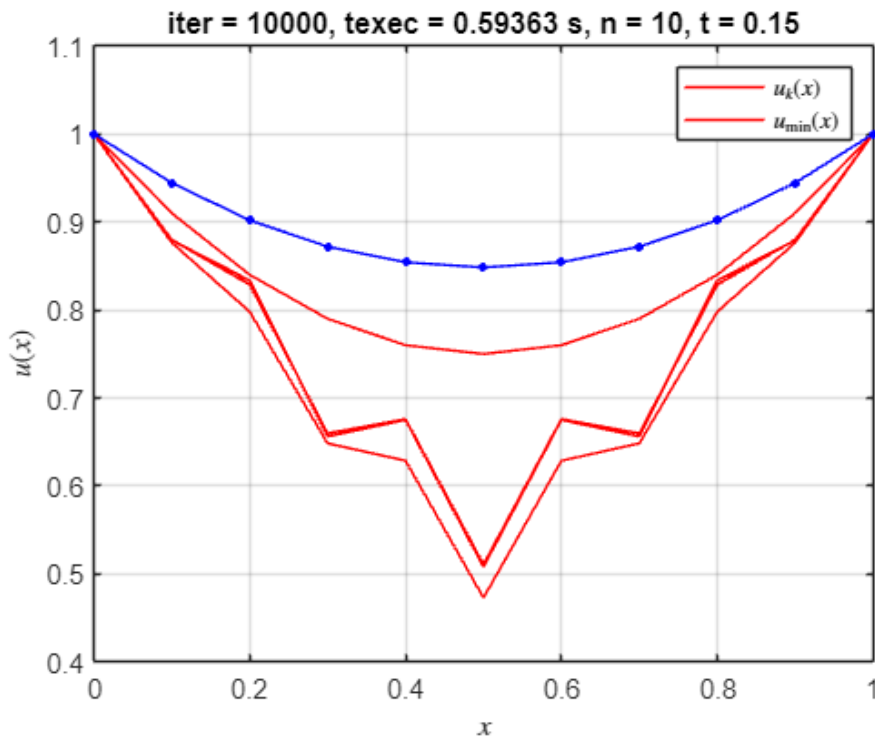


* Parábola: Solución óptima encontrada en la iteración 10000, para $n = 10$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
9997	1.14287899353169	0.258179589445983	1.98910249814077
9998	1.12904143822076	0.258179589445983	2.06033133545535
9999	1.14287899353169	0.258179589445983	1.98910249814077
10000	1.12904143822076	0.258179589445983	2.06033133545535

Valor mínimo de $F(u) = F_{\min_aprox} = 1.142879$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.189255$

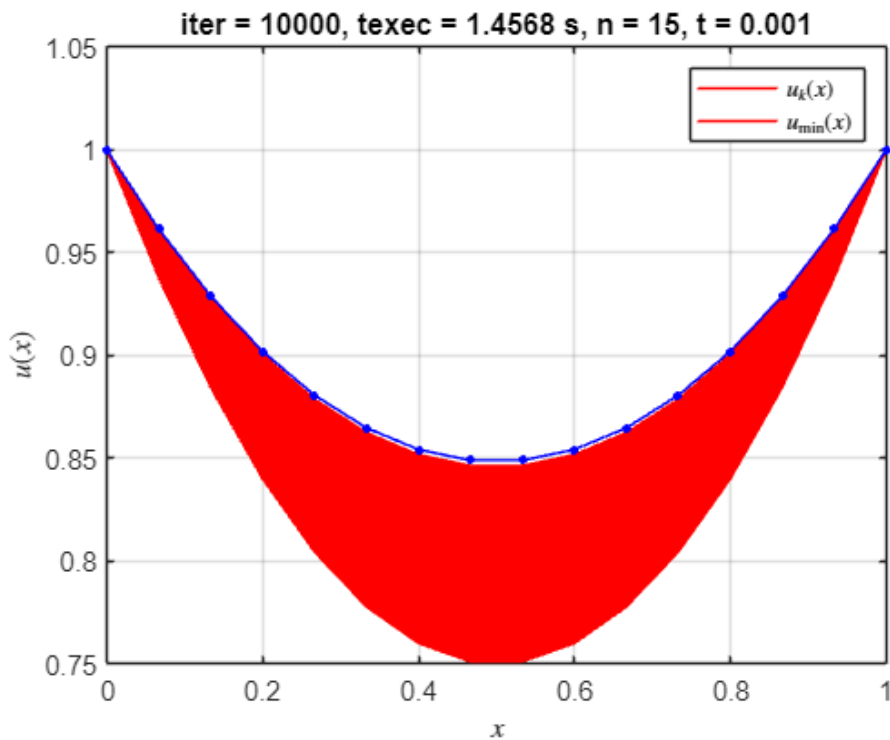


* Parábola: Solución óptima encontrada en la iteración 10000, para $n = 15$, $t = 0.001000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------------

9997	0.953853807887855	2.971024018914e-06	0.757337223354131
9998	0.953853799062561	2.96988692332383e-06	0.757333844455302
9999	0.95385379024402	2.96875025621342e-06	0.757330466838387
10000	0.953853781432229	2.96761401721723e-06	0.757327090502908

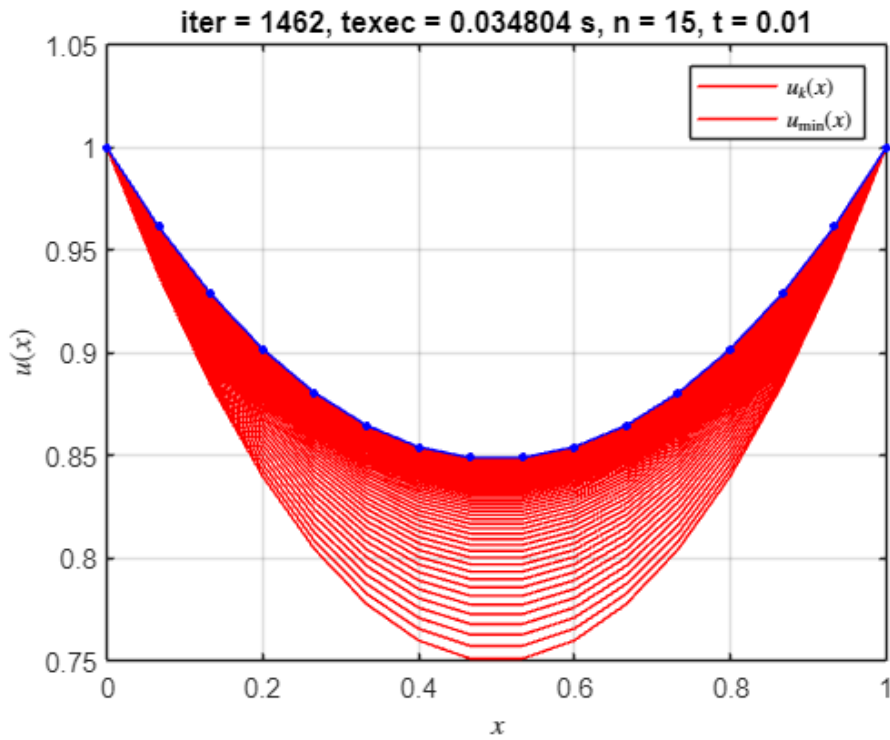
Valor mínimo de $F(u) = F_{\min_aprox} = 0.953854$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000230$



* Parábola: Solución óptima encontrada en la iteración 1462, para $n = 15$, $t = 0.010000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
1459	0.953842660049934	5.02039084302149e-06	0.75001596715527
1460	0.953842657534387	5.00092985511742e-06	0.750010236634153
1461	0.953842655038305	4.98154411350269e-06	0.750004528295484
1462	0.953842652561536	4.96223332882209e-06	0.749998842053738

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953843$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000218$

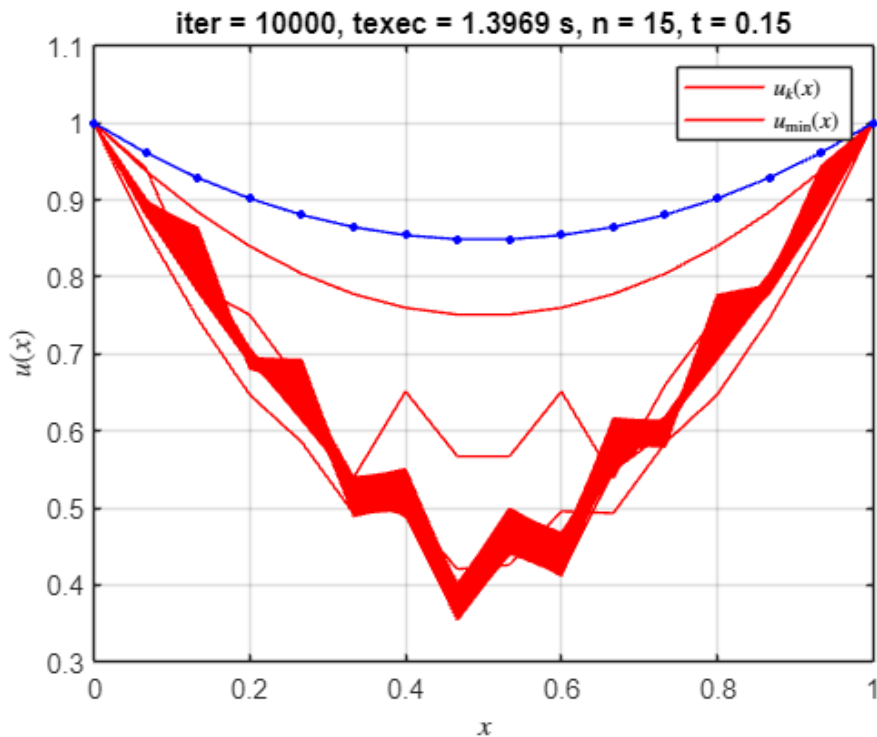


* Parábola: Solución óptima encontrada en la iteración 10000, para n = 15, t = 0.150000

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
9997	1.1339877682629	0.176717402481411	1.56391140612515
9998	1.11567520491267	0.148900843725974	1.5142778048631
9999	1.10283550364693	0.134977435152866	1.54126897987104
10000	1.09742275522515	0.140175939198786	1.63771528731873

Valor mínimo de $F(u) = F_{\min_aprox} = 1.103701$

$|F_{\min_aprox} - I_{\min_exacta}| = 0.150077$

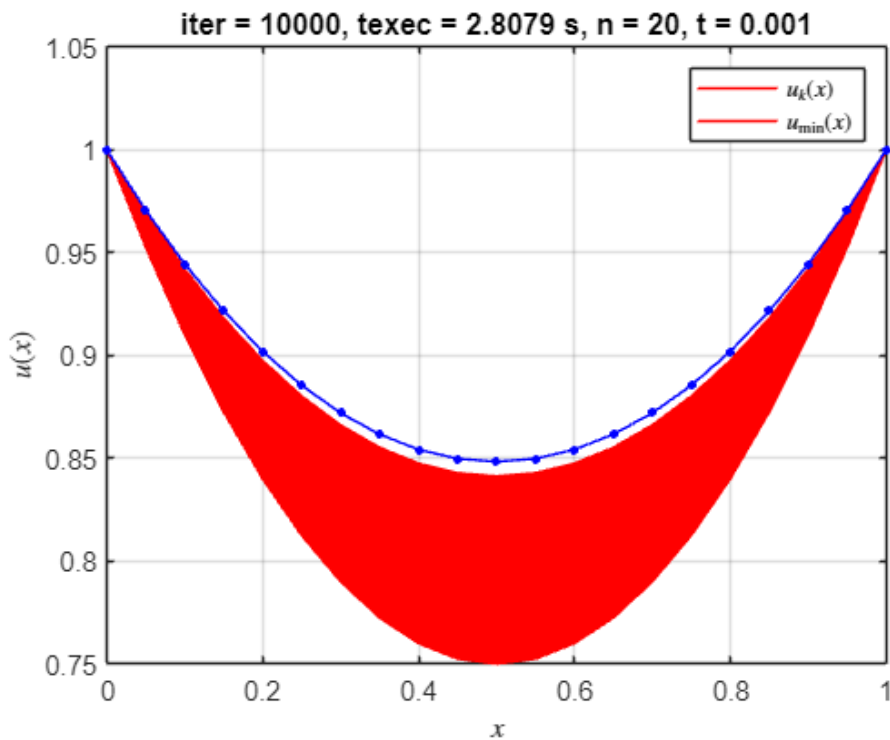


* Parábola: Solución óptima encontrada en la iteración 10000, para n = 20, t = 0.001000

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------------

9997	0.953824059119328	6.62127215493992e-06	0.771432404584388
9998	0.953824015284224	6.61941718429705e-06	0.771425963959607
9999	0.953823971473678	6.61756271154967e-06	0.771419525094748
10000	0.953823927687676	6.61570873651179e-06	0.771413087989363

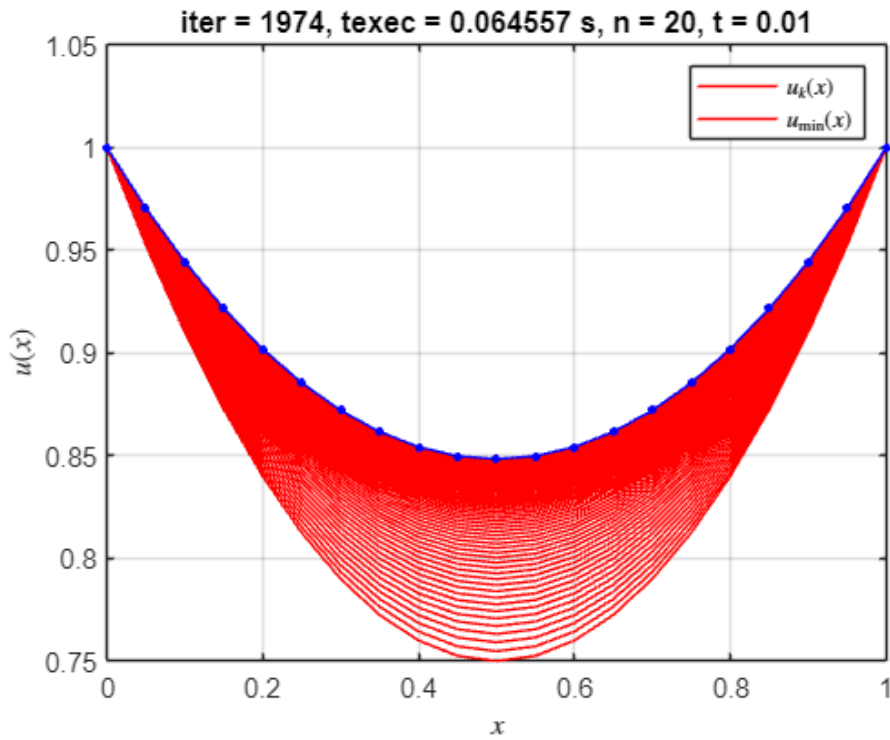
Valor mínimo de $F(u) = F_{\min_aprox} = 0.953824$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000200$



* Parábola: Solución óptima encontrada en la iteración 1974, para $n = 20$, $t = 0.010000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
1971	0.953747154085823	4.02602392740328e-06	0.750008187639733
1972	0.953747152467294	4.01430817404522e-06	0.750004194025355
1973	0.953747150858173	4.00262643315315e-06	0.750000212016589
1974	0.953747149258402	3.99097860629167e-06	0.749996241579833

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953747$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000123$

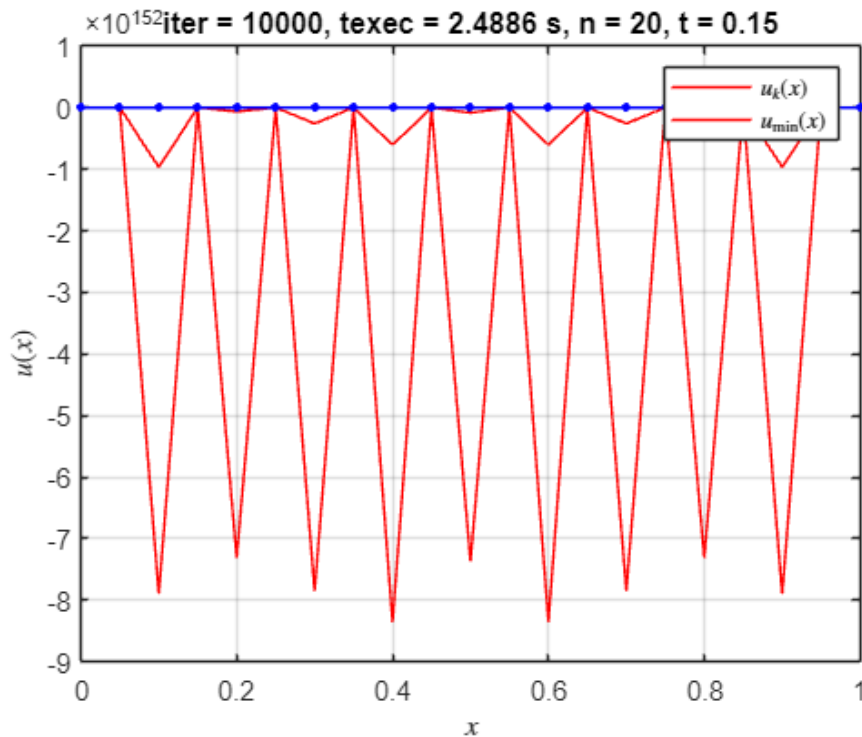


* Parábola: Solución óptima encontrada en la iteración 10000, para $n = 20$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
9997	-Inf	0.152079992293084	1.51607440516857
9998	-Inf	0.152079992293084	1.36826208292915
9999	-Inf	0.152079992293084	1.51607440516857
10000	-Inf	0.152079992293084	1.36826208292915

Valor mínimo de $F(u) = F_{\min_aprox} = -\text{Inf}$

$|F_{\min_aprox} - I_{\min_exacta}| = \text{Inf}$

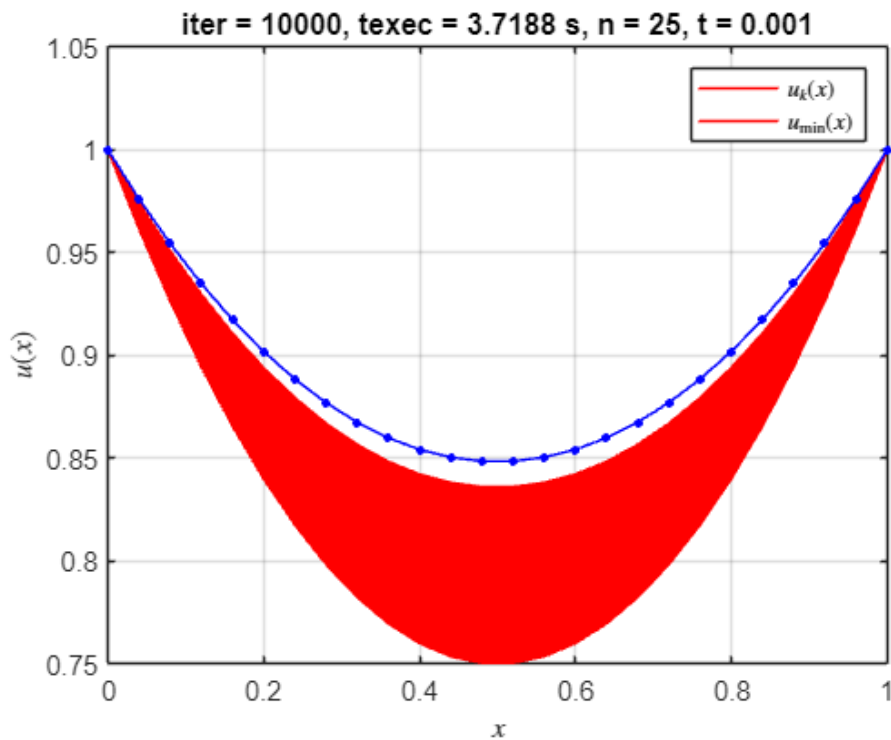


* Parábola: Solución óptima encontrada en la iteración 10000, para $n = 25$, $t = 0.001000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
---	------	-------------------------	----------------------------

9997	0.953940156299898	1.02824633514641e-05	0.78853075337528
9998	0.953940050582327	1.02802302963823e-05	0.788521970437771
9999	0.953939944910667	1.02779976887598e-05	0.788513189319134
10000	0.953939839284902	1.02757655286293e-05	0.788504410019046

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953940$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000316$

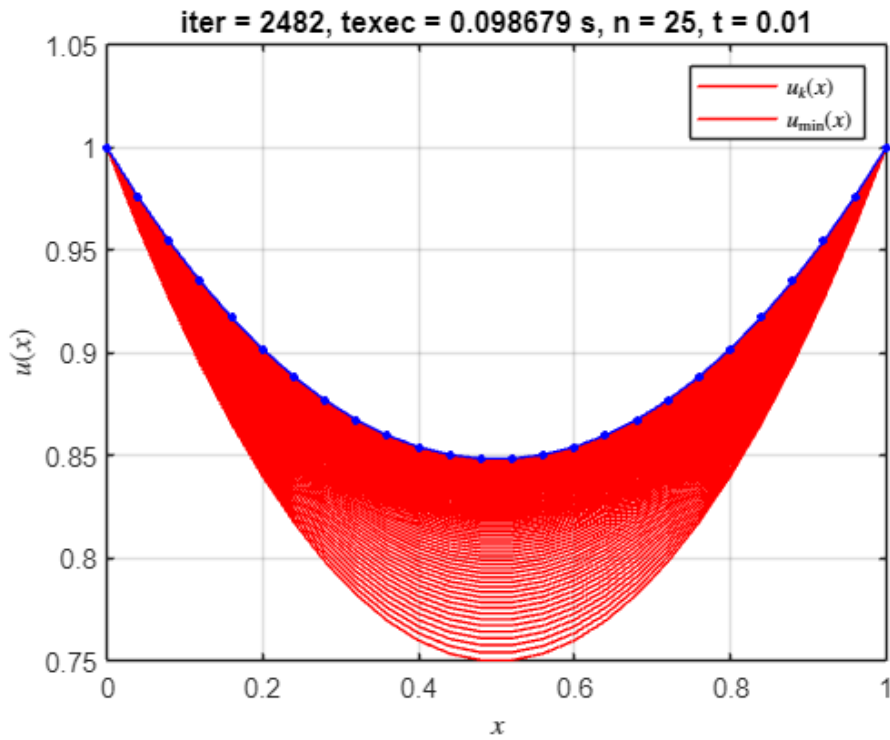


* Parábola: Solución óptima encontrada en la iteración 2482, para $n = 25$, $t = 0.010000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ grad_F(u_{\{k+1\}}) $
---	--------	-------------------------	----------------------------

2479	0.953702942212147	3.47250175984622e-06	0.750006402258543
2480	0.953702941007724	3.46441428671515e-06	0.750003316030701
2481	0.953702939808905	3.45634560669174e-06	0.750000236981014
2482	0.953702938615664	3.44829567577363e-06	0.749997165092847

Valor mínimo de $F(u) = F_{\min_aprox} = 0.953703$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000079$

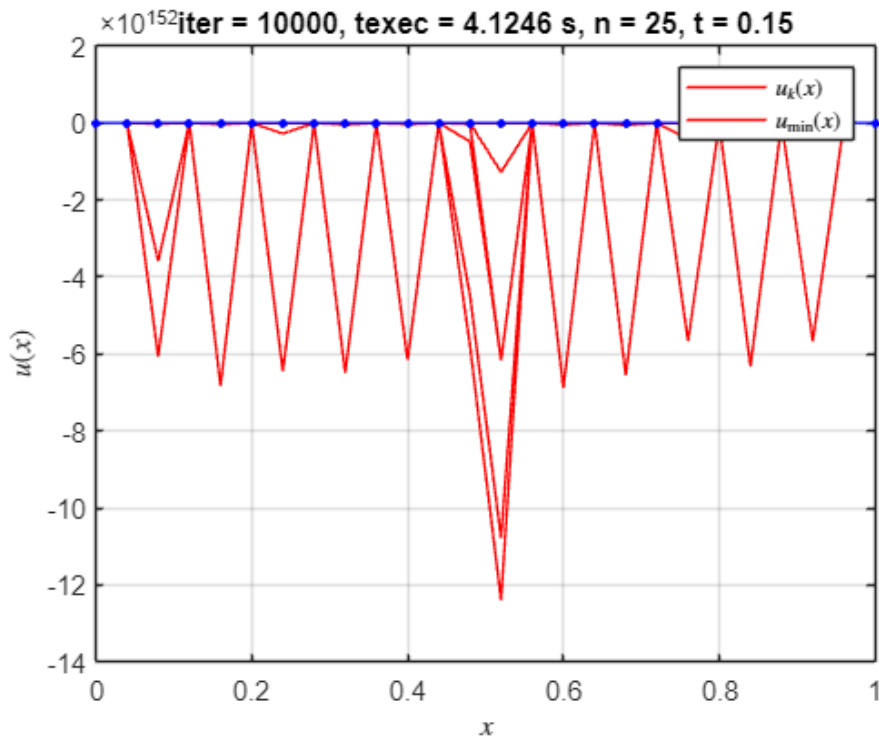


* Parábola: Solución óptima encontrada en la iteración 10000, para n = 25, t = 0.150000

k	F(u)	$ u_{k+1} - u_k $	$ grad_F(u_{k+1}) $
9997	-Inf	0.170947455651523	1.69832500326414
9998	-Inf	0.170947455651523	1.55163417912814
9999	-Inf	0.170947455651523	1.69832500326414
10000	-Inf	0.170947455651523	1.55163417912814

Valor mínimo de F(u) = F_min_aprox = -Inf

$|F_{min_aprox} - I_{min_exacta}| = Inf$

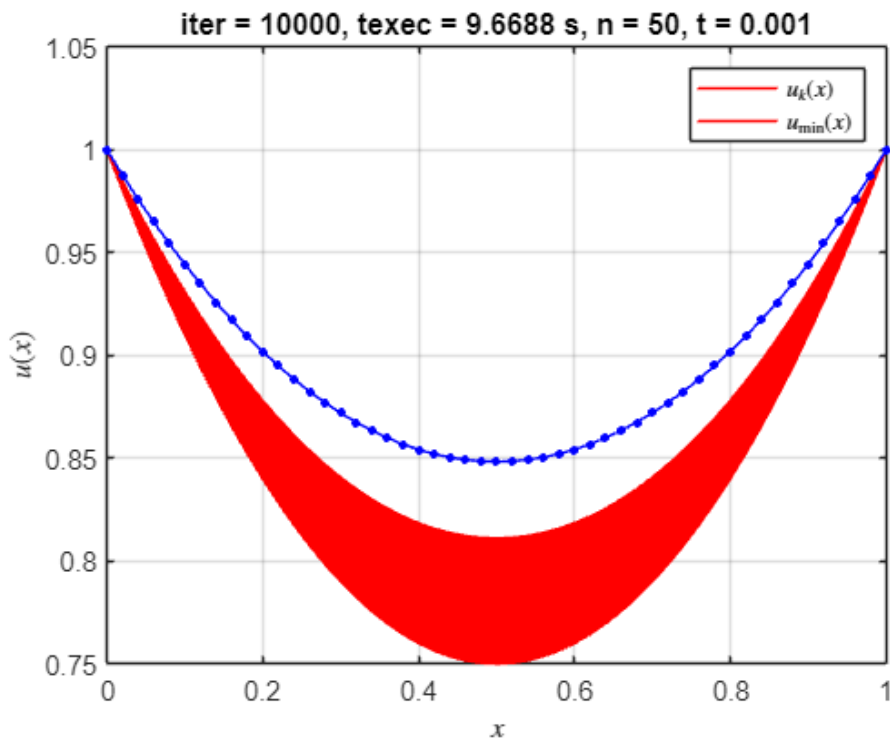


* Parábola: Solución óptima encontrada en la iteración 10000, para n = 50, t = 0.001000

k	F(u)	$ u_{k+1} - u_k $	$ grad_F(u_{k+1}) $
---	------	---------------------	-----------------------

9997	0.955655430905686	2.01432318238515e-05	0.862282127093568
9998	0.955655025174723	2.01413627060786e-05	0.862271080972459
9999	0.955654619519053	2.01394937124713e-05	0.862260035699707
10000	0.955654213938665	2.0137624842932e-05	0.862248991275298

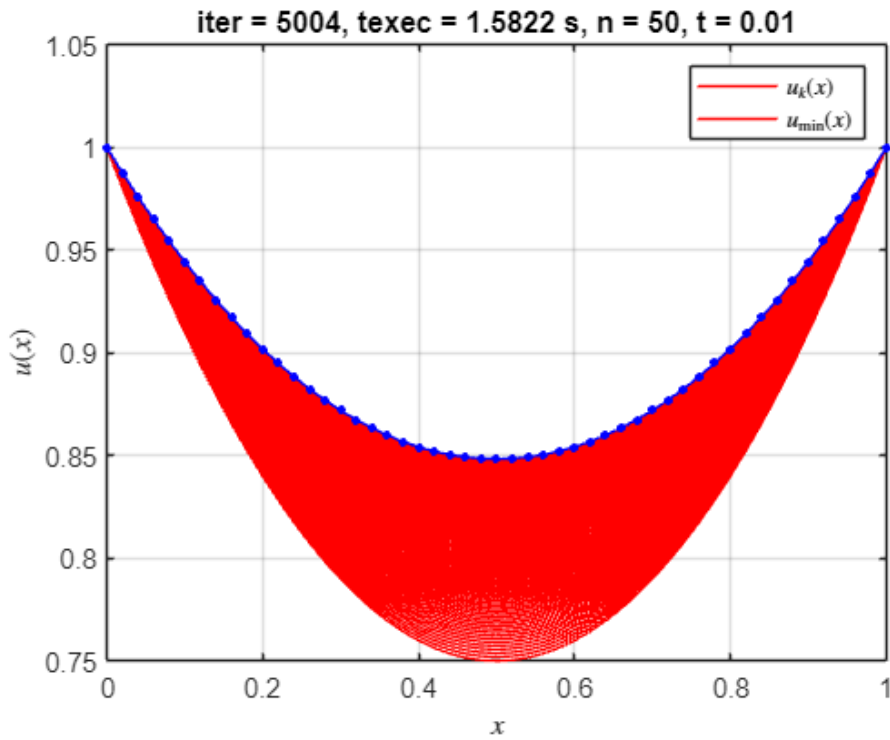
Valor mínimo de $F(u) = F_{\min_aprox} = 0.955654$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.002030$



* Parábola: Solución óptima encontrada en la iteración 5004, para $n = 50$, $t = 0.010000$

k	$F(u)$	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
5001	0.953643986123465	2.33438763646224e-06	0.750004381846178
5002	0.953643985578846	2.33166767859427e-06	0.750002910868014
5003	0.953643985035496	2.32895088315059e-06	0.750001441601482
5004	0.95364398449341	2.32623724624674e-06	0.749999974044598

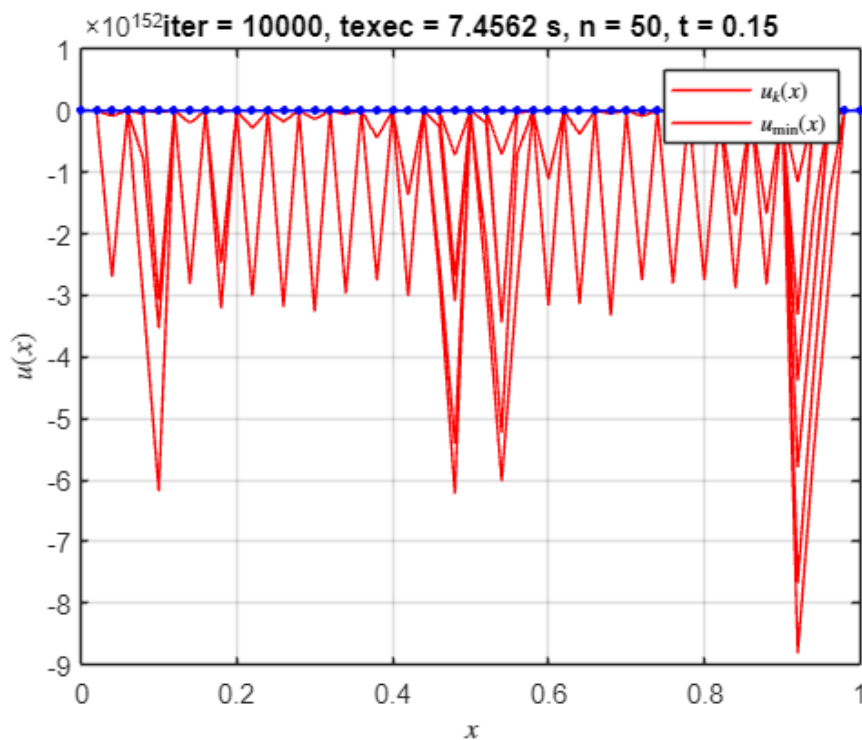
Valor mínimo de $F(u) = F_{\min_aprox} = 0.953644$
 $|F_{\min_aprox} - I_{\min_exacta}| = 0.000020$



* Parábola: Solución óptima encontrada en la iteración 10000, para $n = 50$, $t = 0.150000$

k	F(u)	$ u_{\{k+1\}} - u_k $	$ \text{grad}_F(u_{\{k+1\}}) $
9997	-Inf	0.186786338656144	1.86587795664023
9998	-Inf	0.186786338656144	1.72370729187847
9999	-Inf	0.186786338656144	1.86587795664023
10000	-Inf	0.186786338656144	1.72370729187847

Valor mínimo de $F(u) = F_{\min_aprox} = -\text{Inf}$
 $|F_{\min_aprox} - I_{\min_exacta}| = \text{Inf}$



```
elapsedTime = toc;
```

```
% Muestra el tiempo transcurrido
fprintf('El algoritmo tardó %.4f segundos en ejecutarse.\n', elapsedTime);
```

El algoritmo tardó 7.7986 segundos en ejecutarse.

```
save('resultados_t_fijo.mat','resultados');
```

Método de paso variable

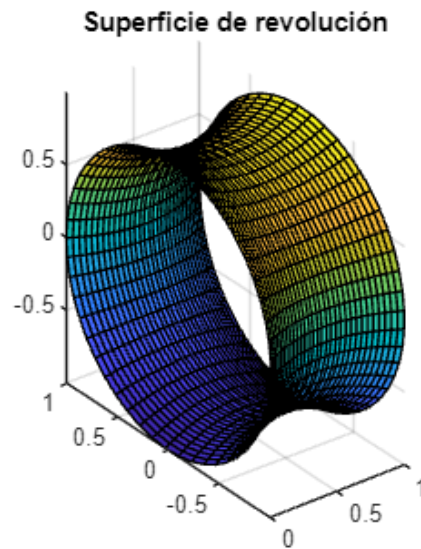
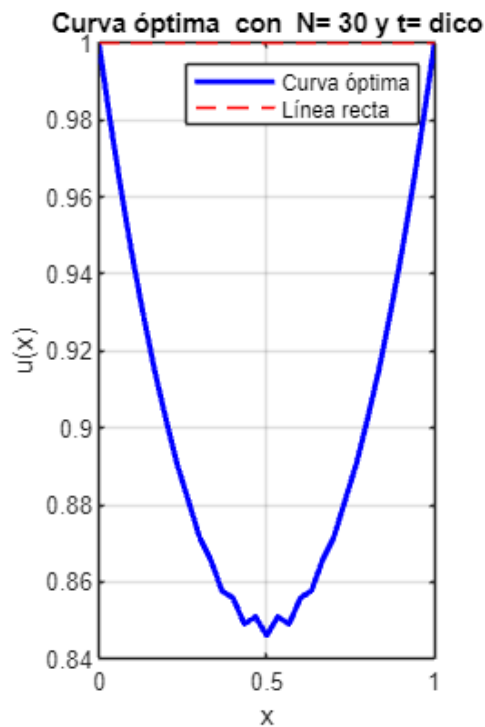
Gradiente con paso variable para N = 30

```
clear, clc, close all;
digits(32)
format shortG
tic;
n = 30; % número de puntos
max_iter = 10000; % máximo de iteraciones
tol = 1e-5; % tolerancia máxima
integral_method = 'trapezium';
t_opt_metod = 'dico';
[u_opt, min_area, alpha_opt] = min_rev_surface(n, max_iter,
tol, integral_method, t_opt_metod);
```

```
=====
SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)
PROGRESO DE OPTIMIZACIÓN - N = 30 PUNTOS
=====
```

Iteración	Área	t	Norma Gradiente
1170	0.927886	0.020138	0.924934
1180	0.927888	0.020138	0.925092
1190	0.927891	0.020146	0.925135
1200	0.927893	0.020146	0.925310
1210	0.927896	0.020154	0.925397
1220	0.927896	0.020146	0.925639
1230	0.927898	0.020146	0.925809
1240	0.927899	0.020146	0.925923
1250	0.927903	0.020154	0.925997
1260	0.927903	0.020154	0.926034

```
-----
```



```
elapsedTime = toc;
% Muestra el tiempo transcurrido
fprintf('El algoritmo tardó %.4f segundos en ejecutarse.\n', elapsedTime);
```

El algoritmo tardó 0.2941 segundos en ejecutarse.

Gradiente con paso variable para distintos puntos

```
clear, clc, close all;
digits(32)
format shortG

% *****
% ***** BENCHMARK *****
%% Con tolerancia constante y 32 dígitos de precisión
tic;
max_iter = 10000; % máximo de iteraciones
tol = 1e-5; % tolerancia máxima
n_values = [5, 10, 15, 20, 25, 50]; % rectas inexactas no tolera n < 15
integral_methods = {'trapezium'};
t_opt_methods = {'fibo', 'dico'};

results = table('Size',
[numel(n_values)*numel(integral_methods)*numel(t_opt_methods), 5], ...
    'VariableTypes', {'double', 'double', 'double', 'double', 'string'}, ...
    'VariableNames', {'n', 'min_area', 't_opt', 'mean_norm_grad',
't_opt_method'});

row = 1;
for n = n_values
    for integral_method = integral_methods
```

```

for t_opt_method = t_opt_methods
    [u_opt, min_area, t_opt, mean_error] = min_rev_surface(n,
max_iter, tol, integral_method{1}, t_opt_method{1});
    results.n(row) = n;
    results.min_area(row) = min_area;
    results.t_opt(row) = t_opt;
    results.mean_norm_grad(row) = mean_error;
    results.t_opt_method(row) = string(t_opt_method{1});
    row = row + 1;
end
end
end

```

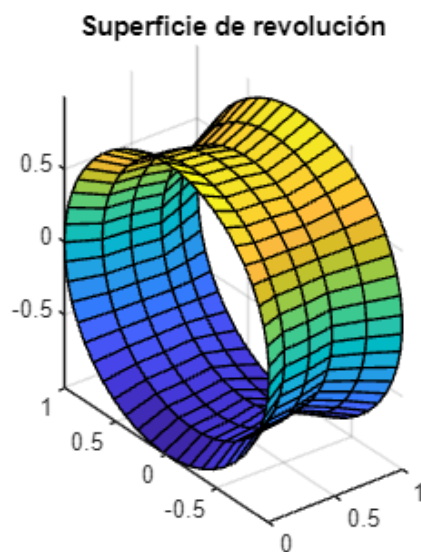
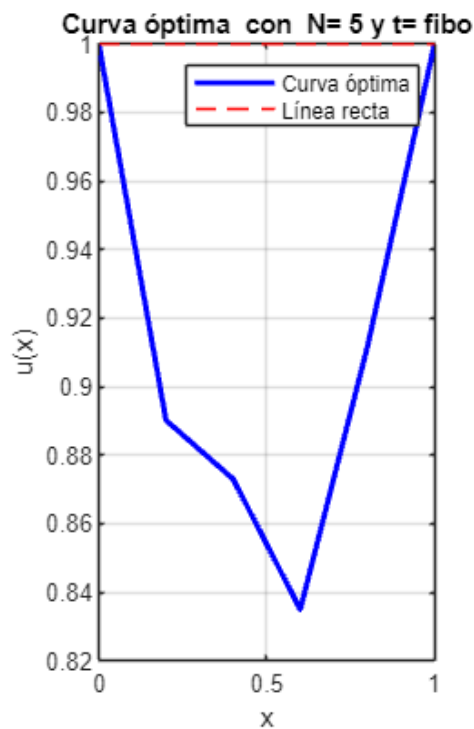
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)

PROGRESO DE OPTIMIZACIÓN - N = 5 PUNTOS

=====

Iteración	Área	t	Norma Gradiente
10	0.824500	0.212244	0.889734
20	0.814971	0.208987	0.760787
30	0.814498	0.198651	0.759526
40	0.814778	0.174435	0.787214
50	0.814990	0.162928	0.807609
60	0.815015	0.162188	0.809382



=====

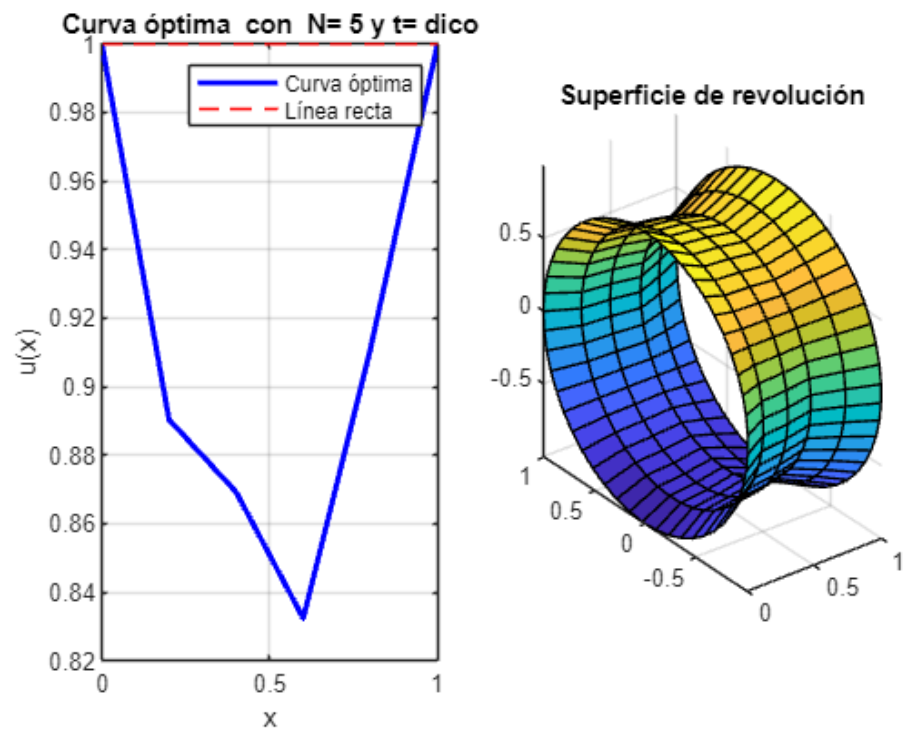
SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)

PROGRESO DE OPTIMIZACIÓN - N = 5 PUNTOS

=====

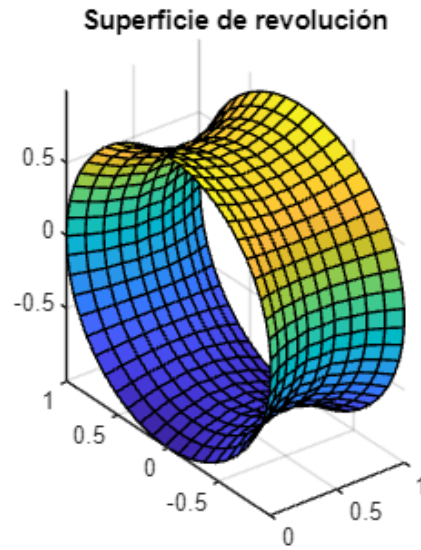
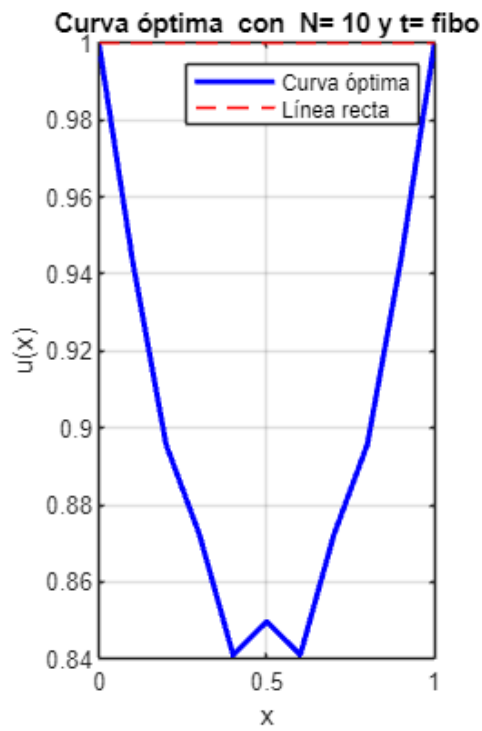
Iteración	Área	t	Norma Gradiente
10	0.817103	0.192379	0.809356
20	0.814940	0.173916	0.790934
30	0.814997	0.163075	0.807530
40	0.815015	0.162205	0.809345
50	0.815016	0.162175	0.809417

60	0.821423	0.131245	0.883605
70	0.822615	0.139599	0.866221
80	0.822714	0.139698	0.867565



=====			
SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)			
PROGRESO DE OPTIMIZACIÓN - N = 10 PUNTOS			
=====			
Iteración	Área	t	Norma Gradiente

10	0.887614	0.061949	1.070117
20	0.885467	0.061210	1.028417
30	0.884070	0.060254	1.006138
40	0.883150	0.059536	0.990471
50	0.882543	0.059058	0.979086
60	0.882138	0.058706	0.971368



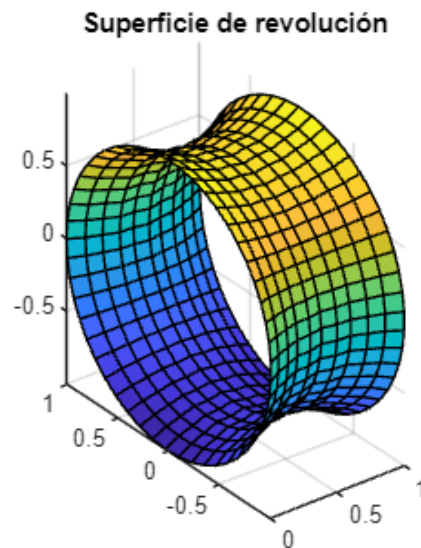
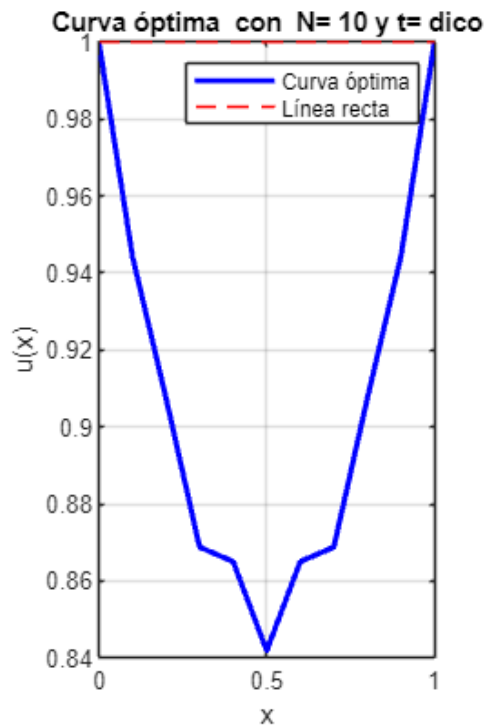
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)

PROGRESO DE OPTIMIZACIÓN - N = 10 PUNTOS

=====

Iteración	Área	t	Norma Gradiente
10	0.878559	0.053975	0.886892
20	0.879302	0.055844	0.900131
30	0.879839	0.056500	0.916648
40	0.880248	0.056950	0.927965

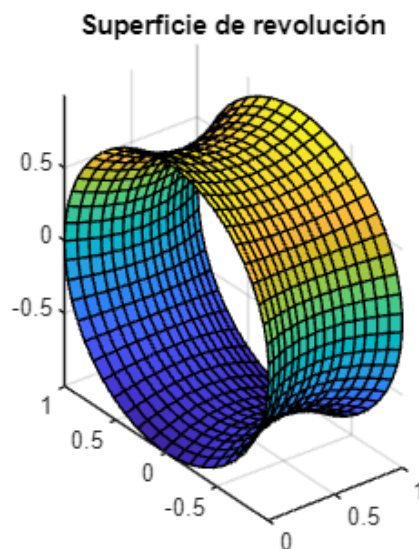
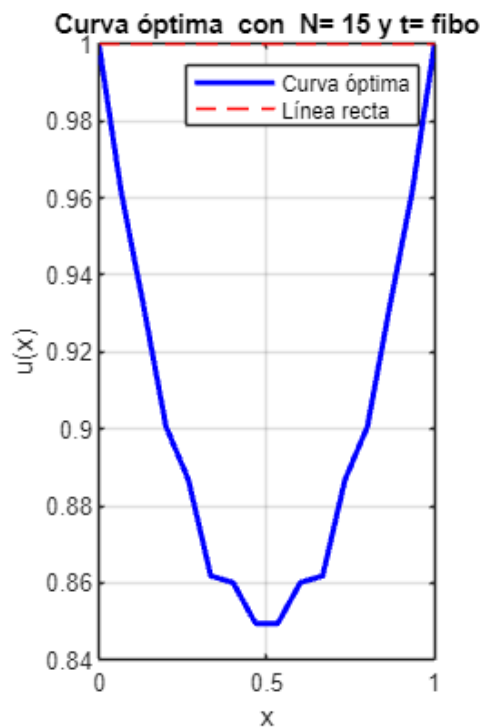


=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)

PROGRESO DE OPTIMIZACIÓN - N = 15 PUNTOS

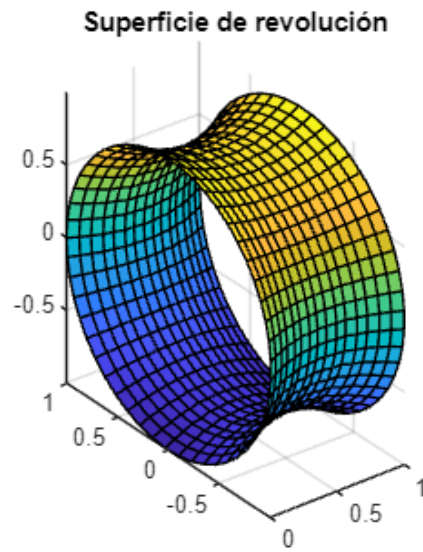
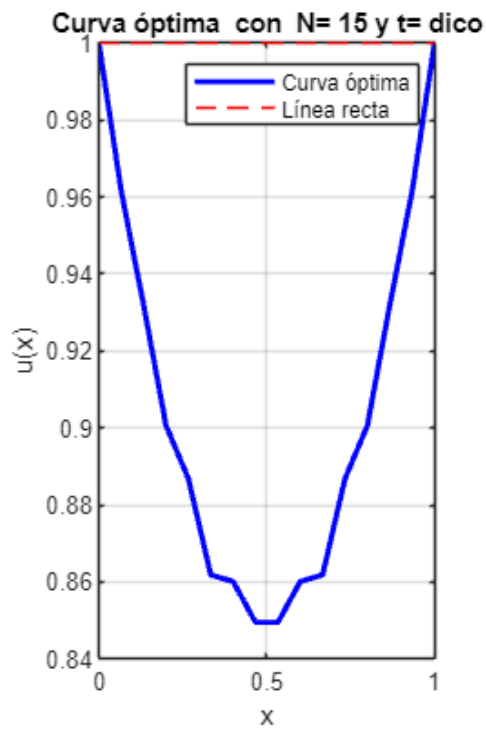
Iteración	Área	t	Norma Gradiente
10	0.902896	0.037783	0.950560
20	0.902311	0.039423	0.882376
30	0.902414	0.039718	0.886217
40	0.902569	0.040014	0.894044
50	0.902719	0.040253	0.901311
60	0.902855	0.040471	0.907130
70	0.902977	0.040641	0.912301
80	0.903092	0.040824	0.916356
90	0.903184	0.040937	0.919962
100	0.903268	0.041063	0.922747



SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)

PROGRESO DE OPTIMIZACIÓN - N = 15 PUNTOS

Iteración	Área	t	Norma Gradiente
70	0.902070	0.038899	0.862638
80	0.902232	0.039326	0.874979
90	0.902401	0.039692	0.885288
100	0.902564	0.039998	0.893858
110	0.902714	0.040249	0.901000
120	0.902851	0.040463	0.907040
130	0.902975	0.040646	0.912049
140	0.903085	0.040806	0.916243
150	0.903181	0.040936	0.919785
160	0.903263	0.041043	0.922738



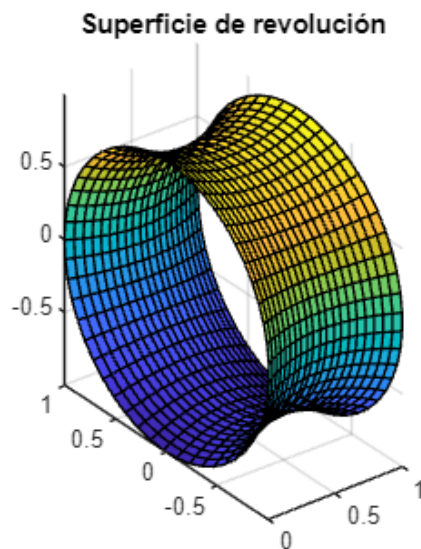
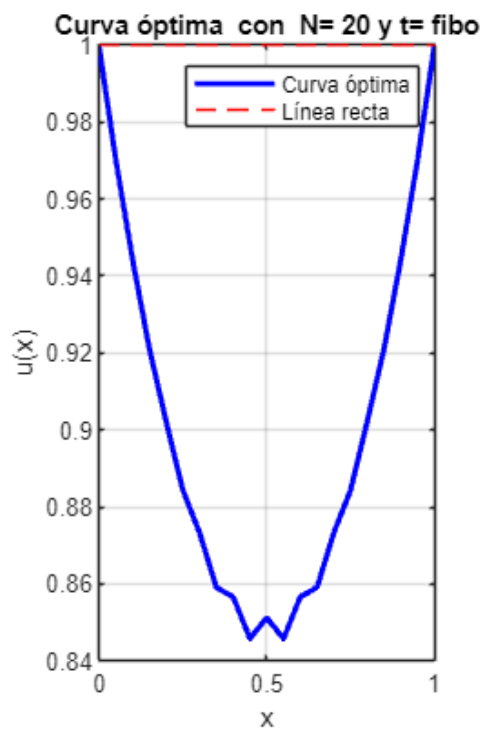
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)

PROGRESO DE OPTIMIZACIÓN - $N = 20$ PUNTOS

=====

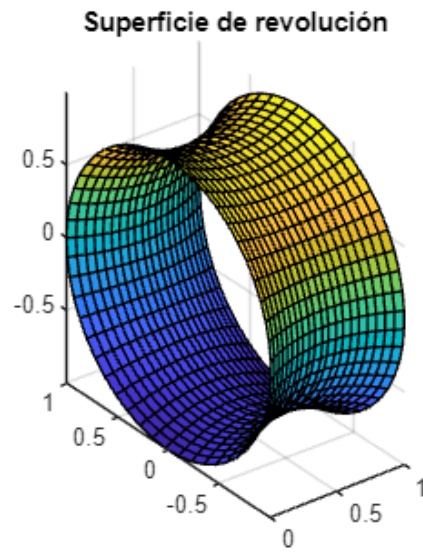
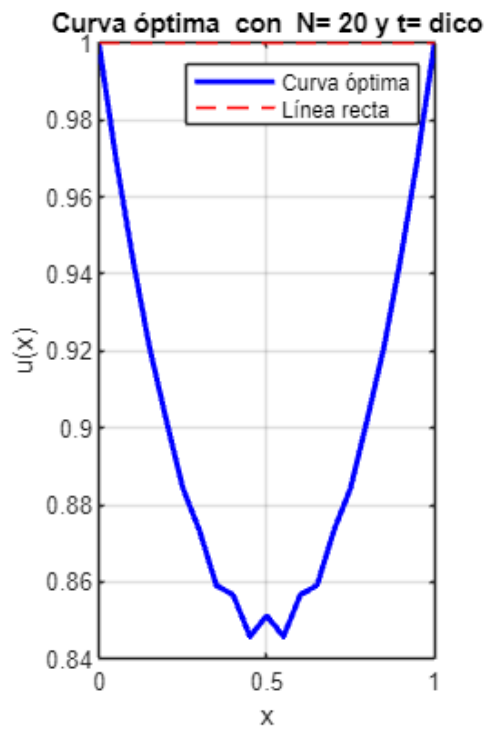
Iteración	Área	t	Norma Gradiente
410	0.915697	0.030389	0.921867
420	0.915715	0.030424	0.922281
430	0.915716	0.030389	0.922994
440	0.915731	0.030424	0.923252
450	0.915737	0.030424	0.923625
460	0.915744	0.030424	0.924042
470	0.915752	0.030445	0.924095
480	0.915756	0.030445	0.924314
490	0.915763	0.030445	0.924719
500	0.915767	0.030445	0.924954



```
=====
SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)
PROGRESO DE OPTIMIZACIÓN - N = 20 PUNTOS
=====
```

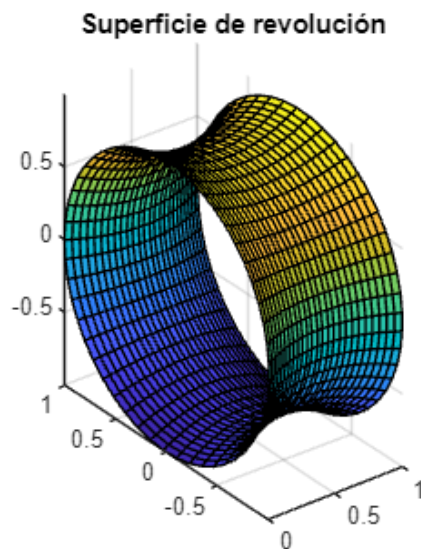
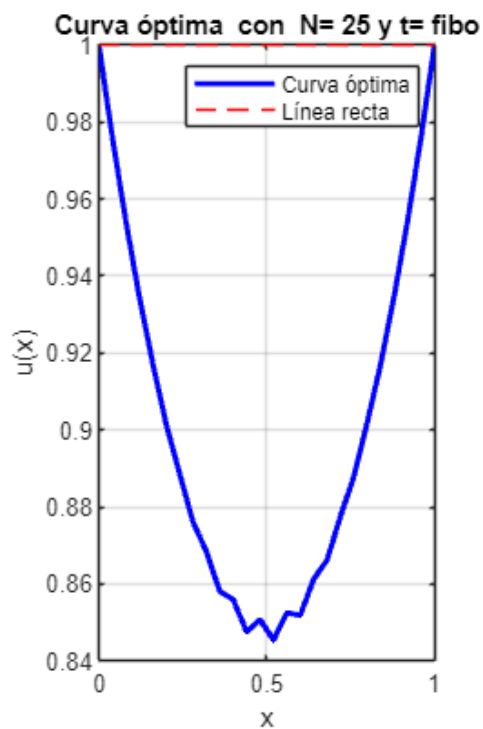
Iteración	Área	t	Norma Gradiente
460	0.915701	0.030400	0.921907
470	0.915710	0.030407	0.922299
480	0.915721	0.030415	0.922800
490	0.915730	0.030423	0.923187
500	0.915736	0.030423	0.923558
510	0.915743	0.030423	0.923982
520	0.915749	0.030430	0.924233
530	0.915757	0.030438	0.924530
540	0.915761	0.030438	0.924779
550	0.915765	0.030438	0.925038

```
-----
```



=====
 SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)
 PROGRESO DE OPTIMIZACIÓN - $N = 25$ PUNTOS
 =====

Iteración	Área	t	Norma Gradiente
710	0.922979	0.024275	0.923796
720	0.922990	0.024310	0.923733
730	0.922990	0.024310	0.923701
740	0.922997	0.024310	0.924228
750	0.922998	0.024310	0.924348
760	0.923006	0.024310	0.924979
770	0.923008	0.024310	0.925172
780	0.923007	0.024310	0.925037
790	0.923013	0.024310	0.925545
800	0.923016	0.024345	0.924784



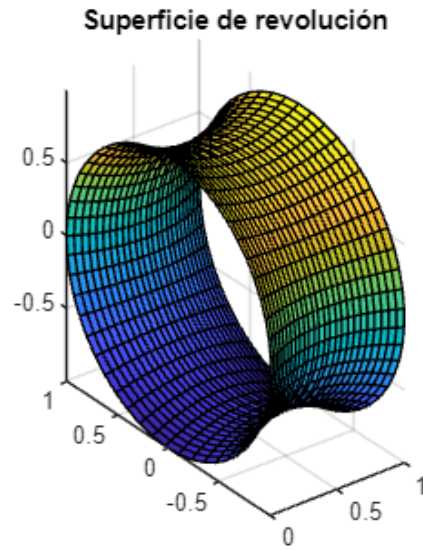
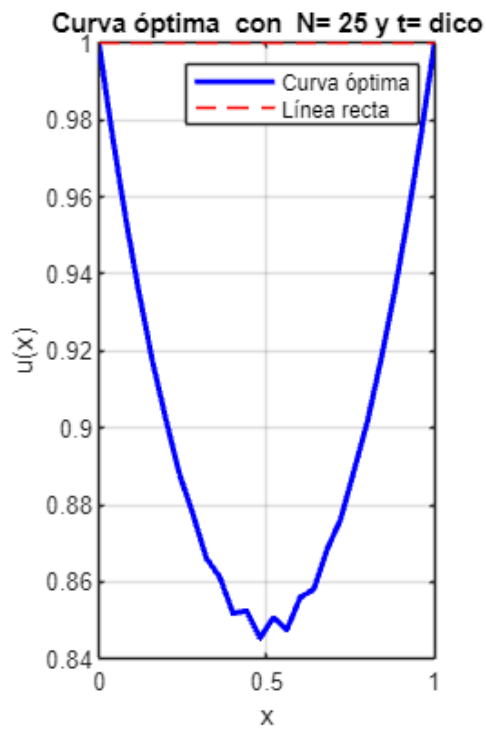
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)

PROGRESO DE OPTIMIZACIÓN - $N = 25$ PUNTOS

=====

Iteración	Área	t	Norma Gradiente
800	0.922990	0.024304	0.923894
810	0.922992	0.024304	0.924093
820	0.922994	0.024304	0.924223
830	0.922998	0.024304	0.924516
840	0.923002	0.024312	0.924596
850	0.923003	0.024312	0.924689
860	0.923007	0.024312	0.925009
870	0.923009	0.024312	0.925173
880	0.923010	0.024312	0.925206
890	0.923014	0.024319	0.925325



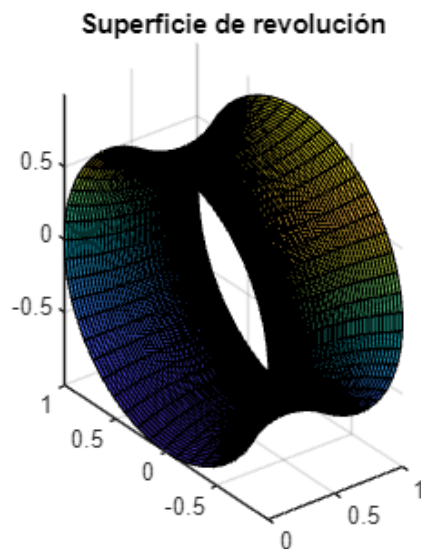
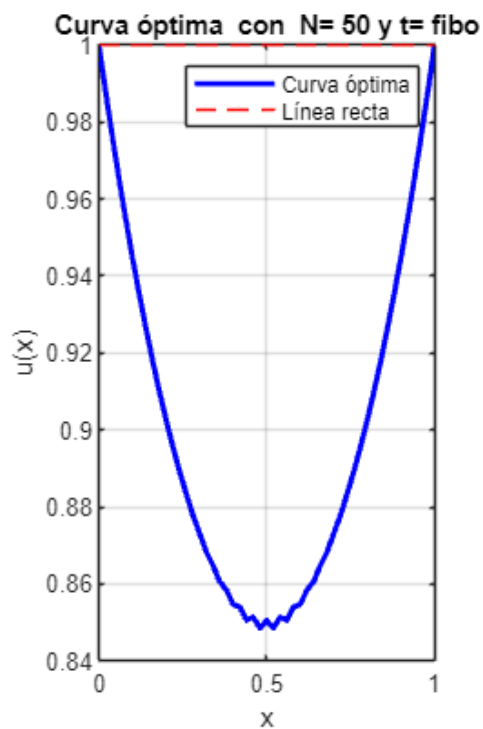
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (FIBO)

PROGRESO DE OPTIMIZACIÓN - $N = 50$ PUNTOS

=====

Iteración	Área	t	Norma Gradiente
2900	0.937914	0.011969	0.923828
2910	0.937916	0.011969	0.924110
2920	0.937916	0.011969	0.924148
2930	0.937915	0.011969	0.923983
2940	0.937920	0.011969	0.924795
2950	0.937916	0.011969	0.924214
2960	0.937919	0.011969	0.924624
2970	0.937920	0.011969	0.924839
2980	0.937920	0.011969	0.924853
2990	0.937919	0.011969	0.924644



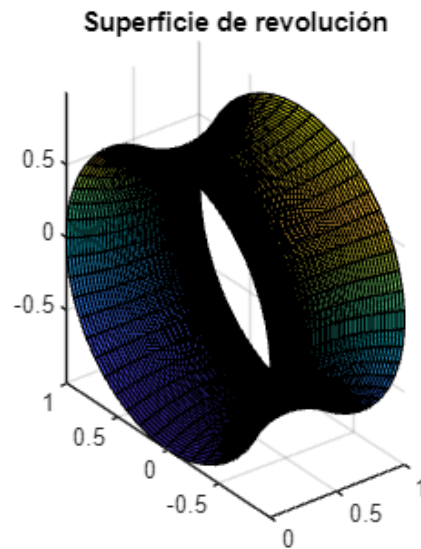
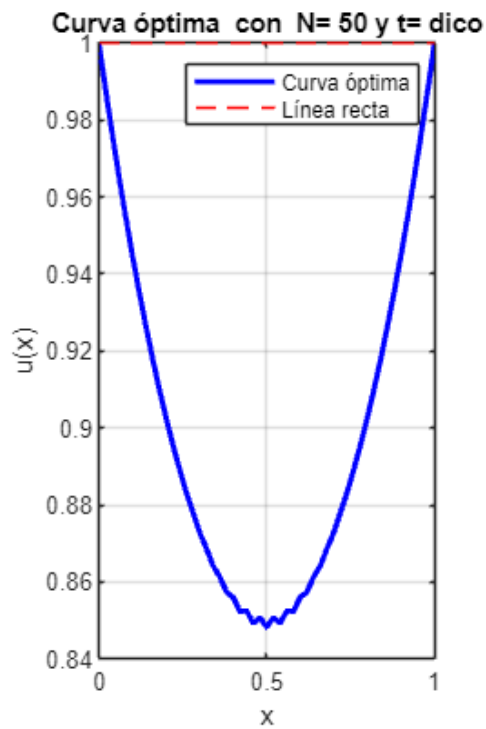
=====

SUPERFICIE MÍNIMA DE REVOLUCIÓN - (DICO)

PROGRESO DE OPTIMIZACIÓN - $N = 50$ PUNTOS

=====

Iteración	Área	t	Norma Gradiente
2970	0.937915	0.011960	0.924460
2980	0.937915	0.011960	0.924559
2990	0.937915	0.011960	0.924441
3000	0.937918	0.011967	0.924534
3010	0.937917	0.011960	0.924829
3020	0.937917	0.011960	0.924910
3030	0.937919	0.011967	0.924799
3040	0.937920	0.011967	0.924909
3050	0.937919	0.011960	0.925227
3060	0.937920	0.011967	0.924938



```
disp(results)
```

n	min_area	t_opt	mean_norm_grad	t_opt_method
5	0.82285	0.13363	0.87646	"fibo"
5	0.82273	0.13971	0.86777	"dico"
10	0.88197	0.070554	0.8939	"fibo"
10	0.8823	0.057209	0.93436	"dico"
15	0.90423	0.041098	0.92384	"fibo"
15	0.90423	0.041081	0.92384	"dico"
20	0.91582	0.030445	0.9249	"fibo"
20	0.91582	0.030446	0.92507	"dico"
25	0.92301	0.024345	0.92488	"fibo"
25	0.92301	0.024312	0.92539	"dico"
50	0.93792	0.011969	0.92485	"fibo"
50	0.93792	0.011967	0.92519	"dico"

```
elapsedTime = toc;
% Muestra el tiempo transcurrido
fprintf('El algoritmo tardó %.4f segundos en ejecutarse.\n', elapsedTime);
```

El algoritmo tardó 0.1258 segundos en ejecutarse.

```
save('resultados_t_variable.mat', "results")
```

Análisis y resultados

Área mínima y el número de puntos N

```
figure;
```

```

% Crear subplots
subplot(1, 2, 1);
hold on;

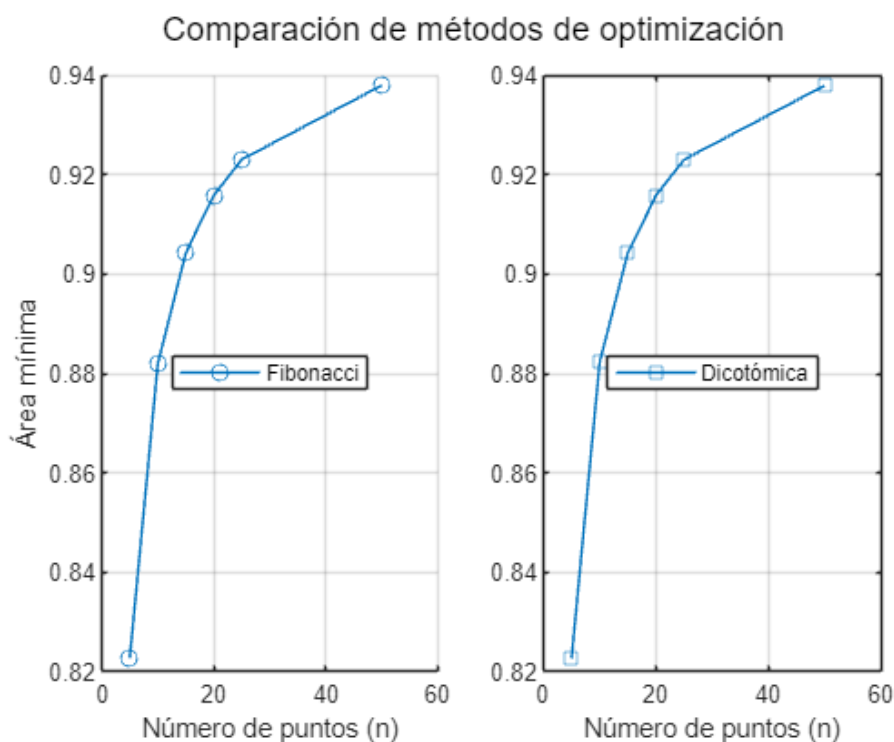
% Graficar min_area vs n para el método 'fibo'
plot(results.n(strcmp(results.t_opt_method, 'fibo')), ...
      results.min_area(strcmp(results.t_opt_method, 'fibo')), ...
      '-o', 'DisplayName', 'Fibonacci');
xlabel('Número de puntos (n)');
ylabel('Área mínima');
% Leyendas
legend('Location', 'best');
grid on;

subplot(1, 2, 2);
% Graficar min_area vs n para el método 'dico'
plot(results.n(strcmp(results.t_opt_method, 'dico')), ...
      results.min_area(strcmp(results.t_opt_method, 'dico')), ...
      '-s', 'DisplayName', 'Dicotómica');

% Etiquetas y título comunes
sgtitle('Comparación de métodos de optimización'); % Título general
xlabel('Número de puntos (n)');
% Leyendas
legend('Location', 'best');
grid on;

hold off;

```



Esta figura anterior se muestran dos gráficos que comparan los métodos de optimización de Fibonacci y búsqueda dicotómica. Ambos gráficos muestran un comportamiento de convergencia similar a medida

que aumenta el número de puntos (N). El mínimo de superficie se acerca a aproximadamente 0,94 cuando n alcanza los 60 puntos.

Ambos métodos muestran una rápida convergencia inicial hasta aproximadamente $n = 20$, después de lo cual la mejora se vuelve más gradual.

El método dicotómico parece tener un comportamiento de progresión ligeramente más suave en comparación con el método de Fibonacci.

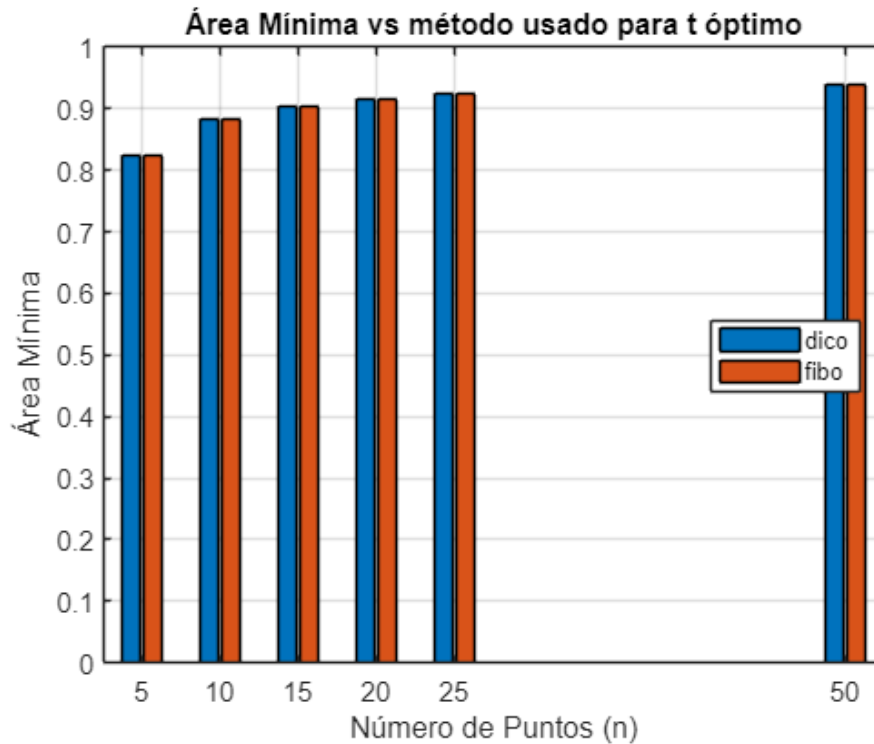
Cálculo del Área mínima y el método de búsqueda de t óptimo

```
% Valores únicos
n_values = unique(results.n);
t_opt_methods = unique(results.t_opt_method);

% Preasignación de matriz con n y método de t óptimo
min_area_matrix = zeros(length(n_values), length(t_opt_methods));

% Rellenar la matriz con datos min_area
for i = 1:length(n_values)
    for j = 1:length(t_opt_methods)
        mask = results.n == n_values(i) & results.t_opt_method ==
t_opt_methods(j);
        min_area_matrix(i, j) = results.min_area(mask);
    end
end

figure;
bar(n_values, min_area_matrix, 'grouped');
xlabel('Número de Puntos (n)');
ylabel('Área Mínima');
title('Área Mínima vs método usado para t óptimo');
legend(t_opt_methods, 'Location', 'best');
grid on;
```

En el gráfico de barras se compara el área mínima obtenida por los dos métodos (Búsqueda dicotómica y Fibonacci) a diferentes números de puntos (5, 10, 15, 20, 25 y 50). Los resultados muestran que ambos métodos logran resultados muy similares en todos los valores de puntos y además que el área mínima aumenta con el número de puntos, estabilizándose alrededor de 0,93-0,94

No se evidencia una diferencia significativa entre el rendimiento de los dos métodos en términos de área mínima final obtenida.

Estudio del perfil del gradiente

Para t fijo

```
r = load("resultados_t_fijo.mat");

figure;
hold on

n_values = unique([r.resultados.n]);
t_values = unique([r.resultados.t]);

for i = 1:length(n_values)
    current_n = n_values(i);
    idx = find([r.resultados.n] == current_n);
    mean_errors = zeros(size(t_values));
    for j = 1:length(t_values)
        idx_nt = find([r.resultados.n] == current_n & [r.resultados.t] ==
t_values(j));
```

```

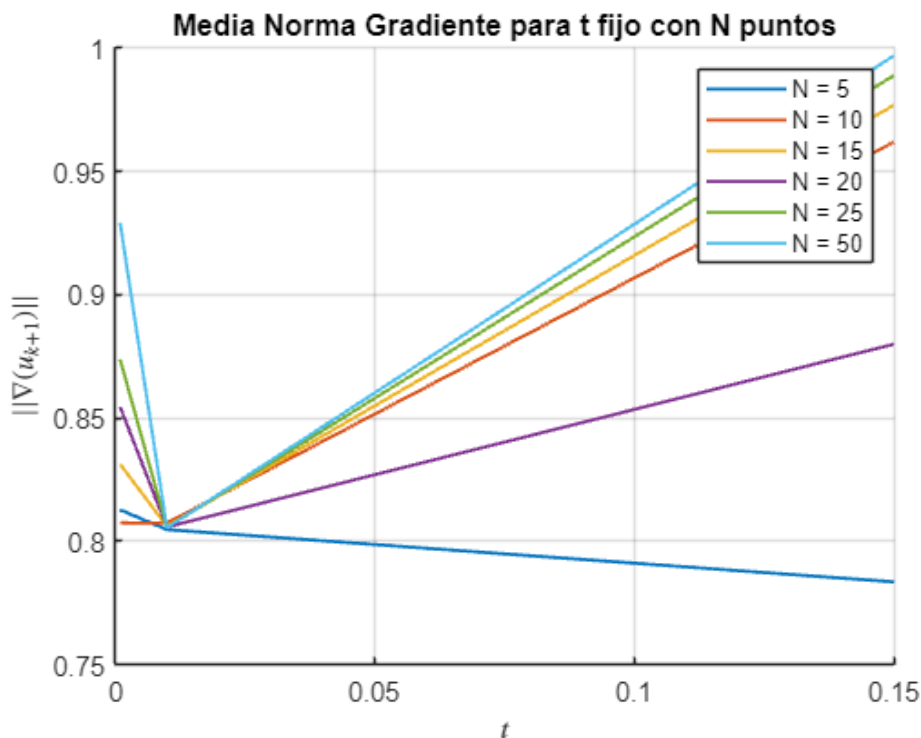
error_data = r.resultados(idx_nt).tabla_iteraciones(:,4);
error_data = error_data(error_data < 1);
if ~isempty(idx_nt)
    mean_errors(j) = mean(error_data);
end
end

plot(t_values, mean_errors, ...
    'LineWidth', 1.2, ...
    'DisplayName', sprintf('N = %d', current_n));
end

xlabel('$$$t$', 'Interpreter', 'latex');
ylabel('$$$||\nabla(u_{k+1})||$', 'Interpreter', 'latex');
title('Media Norma Gradiente para t fijo con N puntos');
grid on;
legend('show');

hold off

```



Este gráfico muestra la evolución de la norma de gradiente para diferentes números de puntos (Desde 5 hasta 50) en comparación con un paso de tamaño fijo t . Todas las curvas muestran un aumento inicial en la norma del gradiente. Además, los valores de N más bajos (especialmente $N = 5$) muestran un comportamiento más estable, pero con valores de N más altos muestran disminuciones más notables en la norma del gradiente para distintos valores de t .

Las curvas divergen considerablemente después de $t \approx 0.05$, lo que sugiere que la elección del tamaño del paso se vuelve más crítica con más puntos.

Para t variable

```
r_variable = load("resultados_t_variable.mat");
t_opt = r_variable.results.t_opt;
mean_norm_grad = r_variable.results.mean_norm_grad;
n = r_variable.results.n;
t_opt_method = r_variable.results.t_opt_method;

unique_methods = unique(t_opt_method);
shapes = {'o', 's'};
colors = [0.8500 0.3250 0.0980; 0.0000 0.4470 0.7410]; % Paleta mejorada
(rojo y azul)

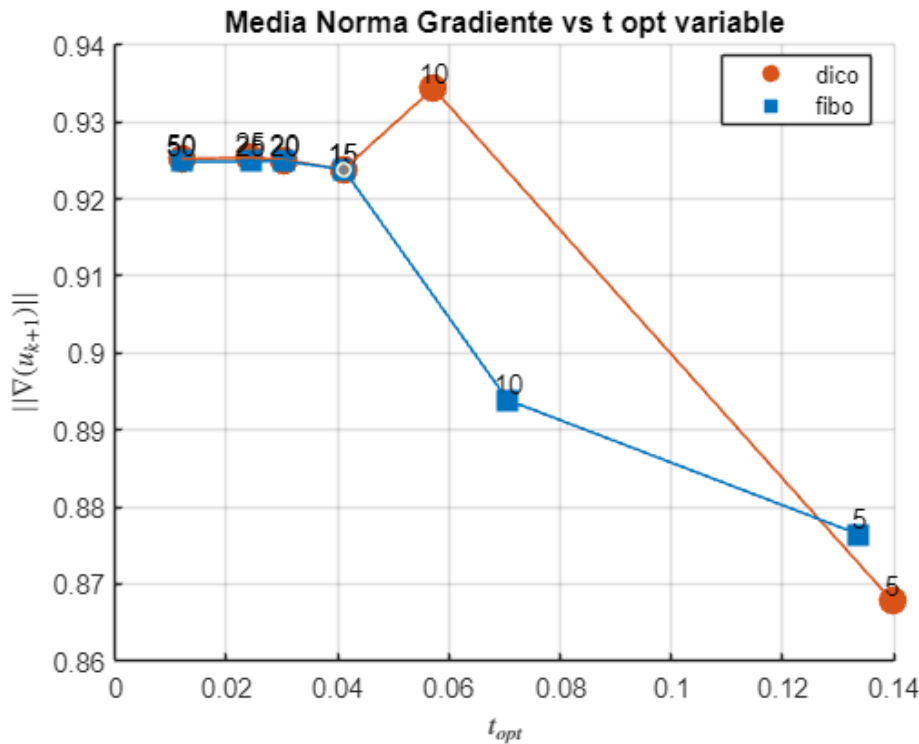
figure;
hold on;

for i = 1:length(unique_methods)
    method_mask = strcmp(t_opt_method, unique_methods{i});
    scatter(t_opt(method_mask), mean_norm_grad(method_mask), 100, ...
        'Marker', shapes{i}, 'MarkerEdgeColor', colors(i, :), ...
        'MarkerFaceColor', colors(i, :), 'DisplayName', unique_methods{i});

    % Añadir etiquetas de n sobre cada punto
    for j = find(method_mask)'
        text(t_opt(j), mean_norm_grad(j), sprintf('%d', n(j)), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center',
            ...
            'FontSize', 10, 'Color', 'k');
    end
end

% Línea de tendencia por método
for i = 1:length(unique_methods)
    method_mask = strcmp(t_opt_method, unique_methods{i});
    plot(t_opt(method_mask), mean_norm_grad(method_mask), ...
        'Color', colors(i, :), 'LineStyle', '-', 'LineWidth', 1,
        'HandleVisibility', 'off');
end

xlabel('$t_{opt}$$', 'Interpreter', 'latex');
ylabel('$||\nabla(u_{k+1})||$', 'Interpreter', 'latex');
title('Media Norma Gradiente vs t opt variable');
legend('Location', 'best');
grid on;
set(gca, 'FontSize', 10);
hold off;
```



El gráfico anterior compara los valores t óptimos para los dos métodos, y, de este se desprende que ambos métodos convergen a valores finales similares alrededor de $t = 0,15$. El método dicotómico (dico) muestra un pico alrededor de $t = 0,05$. También, el método Fibonacci (fibo) muestra una disminución más gradual. Además, se evidencia que ambos métodos funcionan de manera similar para una mayor cantidad de puntos (50, 25, 15) pero con valores de t pequeños.

CONCLUSIONES

La implementación de métodos de descenso de gradiente para encontrar superficies mínimas de revolución demuestra de manera exitosa la aplicación práctica de la optimización multivariada sin restricciones. A través de una exploración sistemática de tamaños de paso fijos y variables, hemos establecido una relación entre los fundamentos teóricos y su implementación computacional. La comparación entre diferentes funciones iniciales y métodos de selección del tamaño de paso proporcionó información valiosa sobre la dinámica de optimización en el ejercicio en cuestión.

Además, esta actividad conecta efectivamente los conceptos introducidos en el tema 2 (búsqueda unidimensional) con desafíos computacionales más complejos, destacando la importancia de una selección de parámetros adecuada y una estrategia de optimización para lograr la convergencia.

La elección del número de puntos (N) tiene un impacto significativo en el comportamiento de optimización, y los valores de N más altos generalmente conducen a soluciones más estables pero más intensivas desde el punto de vista computacional. La selección del tamaño de paso óptimo (t) parece ser más crítica para valores intermedios, mientras que ambos métodos convergen a resultados finales similares.

Finalmente, esta práctica se centró particularmente en el papel de los tamaños de paso adaptativos en la mejora del rendimiento de la optimización, al tiempo que evidencian los requisitos computacionales inherentes a este tipo de problemas.

REFERENCIAS

- Bonnans, J. F., Gilbert, J. C., Lemaréchal, C., & Sagastizábal, C. A. (2021). Numerical optimization: Theoretical and practical aspects. Springer.
- Boyd, S., & Vandenberghe, L. (2020). Convex optimization. Cambridge University Press.
- Nocedal, J., & Wright, S. J. (2022). Numerical optimization (3rd ed.). Springer Science.
- Sun, W., & Yuan, Y. X. (2021). Optimization theory and methods: Nonlinear programming. Springer.

ANEXOS

Anexo I: Método Gradiente Paso fijo

Con aproximación a una recta $u(x) = 1$

```
function [tabla_iteraciones, u, u_hist, k, texec] =  
metodo_gradiente_paso_fijo(n, t, delta_1, delta_2, max_k)  
    k = 0; % índice de iteración  
    error_1 = delta_1 + 1; % error ||u_{k+1} - u_k||  
    error_2 = delta_2 + 1; % error ||grad_f(u_{k+1})||  
    u = ones(n+1, 1); % vector de inicio  
    u_hist = u'; % guarda historial de iteraciones  
    tabla_iteraciones = [];  
    tic;  
    while (error_1 > delta_1 || error_2 > delta_2) && k < max_k  
        k = k + 1;  
        u_nuevo = u - t * grad_F(u);  
        u_nuevo(1) = 1; u_nuevo(n+1) = 1; % exijo las condiciones de contorno!!!  
        error_1 = norm(u_nuevo - u);  
        error_2 = norm(grad_F(u_nuevo));  
        tabla_iteraciones = [tabla_iteraciones; k, F(u), error_1, error_2];  
        u = u_nuevo;  
        u_hist = [u_hist; u'];  
    end  
    texec = toc;  
end
```

Con aproximación a una parábola $u(x) = x^2 - x + 1$;

```
function [tabla_iteraciones, u, u_hist_para, k, texec] =  
metodo_gradiente_paso_fijo_para(n, t, delta_1, delta_2, max_k)  
    k = 0; % índice de iteración  
    error_1 = delta_1 + 1; % error ||u_{k+1} - u_k||  
    error_2 = delta_2 + 1; % error ||grad_f(u_{k+1})||
```

```

x_i = 0:1/n:1;

u = ones(n+1,1);% vector de inicio
for i=2:n

    u(i) = x_i(i)^2-x_i(i)+1;
end
u_hist_para = u'; % guarda historial de iteraciones
tabla_iteraciones = [];
tic;
while (error_1 > delta_1 || error_2 > delta_2) && k < max_k
k = k + 1;
u_nuevo = u - t * grad_F(u);

u_nuevo(1) = 1; u_nuevo(n+1) = 1; % exijo las condiciones de contorno!!!
error_1 = norm(u_nuevo - u);
error_2 = norm(grad_F(u_nuevo));
tabla_iteraciones = [tabla_iteraciones; k, F(u), error_1, error_2];
u = u_nuevo;

u_hist_para = [u_hist_para; u'];
end
texec = toc;
end

```

Anexo II: Función Integral

```

function integral = F(u)
n = length(u) - 1;
integral = 0;
for j = 1:n
integral = integral + (u(j+1)+u(j)) * sqrt(1 + (n*(u(j+1)-u(j))))^2);
end
integral = integral/(2*n);
end

```

Anexo III: Función gradiente

```

function gradiente = grad_F(u)
l = length(u);
n = l - 1;
c = 1/(2*n);
gradiente(1) = (c - u(1)*n*(u(2)-u(1))) / sqrt(1 + (n*(u(2)-u(1))))^2);
for j = 2:n
gradiente(j) = (c + u(j)*n*(u(j)-u(j-1))) / sqrt(1 + (n*(u(j)-u(j-1))))^2)
+ ...
(c - u(j)*n*(u(j+1)-u(j))) / sqrt(1 + (n*(u(j+1)-u(j))))^2);
end
gradiente(1) = (c + u(1)*n*(u(1)-u(1-1))) / sqrt(1 + (n*(u(1)-u(1-1))))^2);
gradiente = gradiente';

```

end

Anexo III: Método Gradiente Paso variable

```
function [u_optimal, min_area, alpha_opt, mean_norm_grad] =  
min_rev_surface(n, max_iter, tol, integral_method, t_opt_method)  
    % Marcando parámetros de entrada  
    if nargin < 1, n = 100; end % número de puntos  
    if nargin < 2, max_iter = 1000; end % máximo de iteraciones  
    if nargin < 3, tol = 1e-4; end % tolerancia por defecto  
    if nargin < 4, integral_method = 'trapezium'; end % Método del trapecio  
por defecto  
    if nargin < 5, t_opt_method = 'fibo'; end % método para encontra t óptimo  
  
    % Inicialización  
    x = linspace(0, 1, n+1);  
    u = ones(1, n+1); % línea recta para y=1  
    iter = 0;  
  
    % Add these arrays to store the last 10 iterations data  
    last_areas = zeros(1, 10);  
    last_alphas = zeros(1, 10);  
    last_norms = zeros(1, 10);  
    last_iters = zeros(1, 10);  
  
    % Método de optimización sin restricciones (método del gradiente con t  
variable)  
    % Se espera implementarlo con otros métodos (Newton y Cuasi-Newton)  
    if strcmp(integral_method, 'trapezium')  
        while iter < max_iter  
            grad = trapezium_gradient(u, n);  
  
            % establecemos función de línea para búsqueda de alpha  
            f = @(alpha) trapezium(u - alpha .* grad, n);  
            df = @(alpha) -sum(grad .* grad); %Sumatoria del vector  
gradiente  
  
            % Llamando a la función Fibonacci o Rectas Inexactas para  
búsqueda de alpha (t) óptimo  
            if strcmp(t_opt_method, 'fibo')  
                [alpha, ~] = fibo(f, df, 0, 1, tol);  
            elseif strcmp(t_opt_method, 'dico')  
                [alpha, ~] = budi(f, df, 0, 1, tol);  
            else  
                [alpha, ~] = rein(f, df, 0, tol);  
            end  
            % Actualizar u a excepción de los puntos u(a) y u(b) que  
deben ser 1  
            u_new = u - alpha .* grad;
```

```

        % disp(abs(norm(u_new(2:end-1)) - norm(u(2:end-1))))

        if iter > 1 && abs(norm(u_new(2:end-1)) - norm(u(2:end-1))) <
tol
            break;
        end

        u(2:end-1) = u_new(2:end-1);
        iter = iter+1;
        area = trapezium(u, n);

        if mod(iter, 10) == 0
            last_areas = [last_areas(2:end), area];
            last_alphas = [last_alphas(2:end), alpha];
            last_norms = [last_norms(2:end), norm(grad)];
            last_iters = [last_iters(2:end), iter];
        end

        end

        fprintf('\n%s\n',
'=====');
        fprintf('SUPERFICIE MÍNIMA DE REVOLUCIÓN - (%s)\n',
upper(t_opt_method));
        fprintf('PROGRESO DE OPTIMIZACIÓN - N = %d PUNTOS\n', n);
        fprintf('%s\n',
'=====');
        fprintf('%-12s %-15s %-15s %-15s\n', 'Iteración', 'Área', 't', 'Norma
Gradiente');
        fprintf('%s\n',
'-----');

        start_idx = find(last_iters > 0, 1);
        if isempty(start_idx), start_idx = 1; end

        for i = start_idx:10
            if last_iters(i) > 0 % Only print if there was an actual
iteration
                fprintf('%-12d %-15.6f %-15.6f %-15.6f\n', ...
                    last_iters(i), last_areas(i), last_alphas(i),
last_norms(i));
            end
        end
        fprintf('%s\n',
'-----');

        u_optimal = u;
        min_area = trapezium(u_optimal, n);

```



```

alpha_opt = alpha;
mean_norm_grad = mean(norm(grad));
% Graficamos
% plot_result(x, u_optimal, t_opt_method);

else
    while iter < max_iter
        grad = simpson_gradient(u, n);

        % establecemos función de línea para búsqueda de alpha
        f = @(alpha) simpson13(u - alpha .* grad, n);
        df = @(alpha) -sum(grad .* grad); %Sumatoria del vector
        gradiente

        % Llamando a la función Fibonacci o Rectas Inexactas para
        búsqueda de alpha (t) óptimo
        if strcmp(t_opt_method, 'fibo')
            [alpha, ~] = fibo(f, df, 0, 1, tol);
        elseif strcmp(t_opt_method, 'dico')
            [alpha, ~] = budi(f, df, 0, 1, tol);
            % alpha = 0.0000001;
        else
            [alpha, ~] = rein(f, df, 0,tol);
        end
        % Actualizar u a excepción de los puntos u(a) y u(b) que deben
        ser 1
        u_new = u - alpha .* grad;
        u(2:end-1) = u_new(2:end-1);

        iter = iter +1;
        new_area = simpson13(u, n);

        if iter > 0 && abs(new_area - prev_area) < tol
            break;
        end
        prev_area = new_area;

        if mod(iter, 10) == 0
            if iter == 10
                fprintf('\n%s\n',
'=====');
                fprintf('SUPERFICIE DE REVOLUCIÓN MÍNIMA (%s)\n',
upper(t_opt_method));
                fprintf('%s\n',
'=====');
                fprintf('%-12s %-15s %-15s %-15s\n', 'IteraCion',
'área', 't', 'Norma Gradiente');
                fprintf('%s\n',
'-----');
            end

```

```

        fprintf('%-12d %-15.6f %-15.6f %-15.6f\n', iter,
new_area, alpha, norm(grad));
        if mod(iter, 1000) == 0
            fprintf('%s\n',
'-----');
        end
    end
    end
    u_optimal = u;
    min_area = simpson13(u_optimal, n);
    alpha_opt = alpha;
    % Graficamos
    plot_result(x, u_optimal);
end
plot_result(x, u_optimal, t_opt_method);
end

% Gradiente
function grad = trapezium_gradient(u, n)
    grad = zeros(1, n+1);
    grad(1) = (1/(2*n)) * (sqrt(1 + n^2*(u(2) - u(1))^2) - ...
        (n^2*(u(2) - u(1))*(u(2) + u(1)))/sqrt(1 + n^2*(u(2) -
u(1))^2));

    % k = 1,2,...,n-1
    for k = 2:n
        grad(k) = (1/(2*n)) * (sqrt(1 + n^2*(u(k) - u(k-1))^2) + ...
            (n^2*(u(k) - u(k-1))*(u(k) + u(k-1)))/sqrt(1 + n^2*(u(k) -
u(k-1))^2) + ...
            sqrt(1 + n^2*(u(k+1) - u(k))^2) - ...
            (n^2*(u(k+1) - u(k))*(u(k+1) + u(k)))/sqrt(1 + n^2*(u(k+1) -
- u(k))^2));
    end

    % k = n
    grad(n+1) = (1/(2*n)) * (sqrt(1 + n^2*(u(n+1) - u(n))^2) + ...
        (n^2*(u(n+1) - u(n))*(u(n+1) + u(n)))/sqrt(1 + n^2*(u(n+1) -
u(n))^2));
end

% Área bajo la curva método del trapezio
function area = trapezium(u, n)
    area = 0;
    for j = 1:n
        Tj = (u(j) + u(j+1)) * sqrt(1 + (n+1)^2 * (u(j+1) - u(j))^2) /
(2*(n+1));
        area = area + Tj;
    end
end

function grad = simpson_gradient(u, n)
    grad = zeros(1, n+1);

```

```

% g0 (primer punto, solo contribución inicial)
grad(1) = (1/(6*n))*(sqrt(1 + n^2*(u(2) - u(1))^2) - ...
          u(1)*n^2*(u(2) - u(1))/sqrt(1 + n^2*(u(2) - u(1))^2));

% Para puntos internos (k = 2 hasta n)
% Cada punto recibe contribución de tres términos Sj consecutivos
for k = 2:n
    % Contribución como punto final del primer término
    term1 = (1/(6*n))*(sqrt(1 + n^2*(u(k) - u(k-1))^2) + ...
            5*sqrt(1 + n^2*(u(k+1) - u(k))^2));

    % Contribución del término con diferencia hacia adelante
    term2 = (1/(6*n))*(n^2*(u(k+1) - u(k))* ...
            (u(k+1) - 0)/sqrt(1 + n^2*(u(k+1) - u(k))^2));

    % Contribución del término con diferencia hacia atrás
    term3 = (1/(6*n))*(n^2*(u(k) - u(k-1))* ...
            (0 - u(k-1))/sqrt(1 + n^2*(u(k) - u(k-1))^2));

    grad(k) = term1 + term2 + term3;
end

% gn (último punto, solo contribución final)
grad(n+1) = (1/(6*n))*(sqrt(1 + n^2*(u(n+1) - u(n))^2) + ...
            4*u(n)*n^2*(u(n+1) - u(n))/sqrt(1 + n^2*(u(n+1) - u(n))^2));

return;
end

% Función para calcular el área usando Simpson
function area = simpson13(u, n)
    area = 0;
    for j = 0:n-2
        fj = u(j+1)*sqrt(1 + n^2*(u(j+2) - u(j+1))^2);
        fj1 = 4*u(j+2)*sqrt(1 + n^2*(u(j+3) - u(j+2))^2);
        fj2 = u(j+3)*sqrt(1 + n^2*(u(j+3) - u(j+2))^2);
        Sj = (1/(6*n))*(fj + fj1 + fj2);
        area = area + Sj;
    end
end

function plot_result(x, u, t_opt_method)
    figure;
    subplot(1,2,1)
    plot(x, u, 'b-', 'LineWidth', 2);
    hold on;
    plot([0,1], [1,1], 'r--'); % Línea recta original
    grid on;
    title(['Curva óptima con N= ' num2str(length(x) - 1) ' y t= '
t_opt_method]);

```

```

xlabel('x');
ylabel('u(x)');
legend('Curva óptima', 'Línea recta');
% Graficar superficie de revolución
subplot(1,2,2)
theta = linspace(0, 2*pi, 50);
[X, T] = meshgrid(x, theta);
U = repmat(u, length(theta), 1);
Y = U .* cos(T);
Z = U .* sin(T);
surf(X, Y, Z);
title('Superficie de revolución');
axis equal;
grid on;

```

```
end
```

Anexo IV: Métodos de búsqueda para t óptimo

Fibonacci

```

function [x_min,f_min,k, prec, lapse] = fibo(fun,dfun,a,b,tol)
    I0 = [a, b];
    % F0 = fun(I0);
    F0 = [fun(I0(1)), fun(I0(2))];
    w1 = I0(2) - I0(1);
    max_iter = 50;
    epsilon = 1e-6;

    tic;
    % Fibonacci serie implementation
    F = zeros(max_iter, 1);
    F(1) = 1; F(2) = 1; n = 2;

    while w1 > tol*F(n)
        n = n+1;
        F(n) = F(n-1) + F(n-2);

    end

    % Fibonacci search algorithm
    w = w1*(F(n-2)/F(n-1));
    xa = I0(2) - w;
    xb = I0(1) + w;
    fa = fun(xa);
    fb = fun(xb);
    k = 0;

    for n=2:n-2
        w = w*(F(n-k-1)/F(n-k));
        if fa > fb

```

```

        I0(1) = xa; F0(1) = fa; xa = xb; fa = fb; xb = xa + w; fb =
fun(xb);
    else
        I0(2) = xb; F0(2) = fb; xb = xa; fb = fa; xa = xb - w; fa =
fun(xa);
    end

    width_interval = abs(I0(2) - I0(1));

    if width_interval < tol
        break;
    end

end
x_min = (I0(2) + I0(1))/2;
f_min = fun(x_min);
k = n;
prec = abs(dfun(x_min));
lapse = toc;
lapse = vpa(lapse);
end

```

Búsqueda Dicotómica

```

function [x_min,f_min,k_opt,prec, lapse] = budi(fun,dfun, a, b, tol)
epsilon = 1e-6;
max_iter = 50;
k = 0;

tic;
x1 = (a+b)/2;
xa = x1-(epsilon/2);
xb = x1 + (epsilon/2);

while abs(b-a) >= tol && k < max_iter

    if fun(xa) <= fun(xb)
        b=xb; a=a;
    elseif fun(xa) > fun(xb)
        a=xa; b=b;
    end

    x1 = (a + b) /2;
    xa = x1-(epsilon/2);
    xb = x1 + (epsilon/2);
    fl = fun(a);
    fr=fun(b);

    k = k+1;

end
x_min = (a+b)/2;
f_min = fun(x_min);

```

```
k_opt = k;  
prec = abs(dfun(x_min));  
lapse = toc;  
lapse = vpa(lapse);  
end
```